

THE PRACTICAL CLAUDE PLAYBOOK

The Practical Claude Playbook

From Your First Chat to Autonomous Systems (2026)

Why this guide exists

Most people use Claude like a smarter search box. They ask it a question, copy the answer, and move on. That works, and it also leaves about 90 percent of the value on the table.

The people getting real leverage from Claude in 2026 are not better at writing prompts. They understand one thing the casual user does not: Claude is not a chatbot. It is an engine that can read your files, run tools, build deliverables, write and ship code, and do all of it on a schedule while you do something else.

This guide takes you from typing your first question to running systems that do real work without you in the loop. You do not need to code to start, and you will have a clear path for the day you want to go further.

Who this is for

Founders, operators, and business owners who want Claude to actually move the needle, not just answer questions. If you run a company, manage a team, or carry the operational load yourself, this is written for you. No engineering background assumed.

How to use this guide

It is built in three tiers. Jump to where you are.

TIER	YOU ARE HERE IF...	YOU WILL LEAVE ABLE TO...
Part 1 - Beginner	You have used Claude a little, or not at all	Get your first real wins this week and understand the whole landscape
Part 2 - Intermediate	You use Claude regularly but stick to chat	Pick the right tool every time and build workflows you reuse
Part 3 - Advanced	You want Claude doing work unattended	Automate operations and know what is worth building yourself vs handing off

One note on timing. Claude's products move fast. Everything here was accurate as of mid-2026. Where a price or limit matters to a decision, confirm the current number before you commit.

The one idea to take from this entire guide

One engine, many doors.

Anthropic builds the intelligence, the underlying models that do the thinking. Then it ships that same intelligence behind different doors, each one shaped for a different kind of person and a different level of independence.

- **The chat door:** For thinking out loud with a human steering every step.
- **The Claude Code door:** For engineers working inside a codebase.
- **The Cowork door:** For non-technical operators who want work done on their own desktop, no terminal required.
- **The API door:** For developers building Claude into their own products and automations.

Same brain. Different hands. The model answering you in a chat window is the same one refactoring a codebase or building a spreadsheet on someone's desktop.

Once this clicks, every "which tool should I use?" question answers itself. You only ever ask two things:

1. Who is doing the work? You, in the loop? An engineer? A non-technical teammate? Your own software?
2. How much autonomy does the task need? A single answer you review, or a multi-step job that runs on its own?

Hold those two questions in your head as you read. The rest of this guide is just teaching you to answer them well.

CONTENTS

Table of Contents

Three levels, thirty sections, one climb.

Beginner		PAGE
01	What Claude Actually Is (and Isn't)	5
02	The Claude Universe: Every Way to Use It	7
03	Claude.ai Deep Dive: Your First Real Workspace	10
04	Claude on Your Desktop and Phone	13
05	The Beginner's Prompting Playbook	15
06	10 Everyday Things to Use Claude For	17
07	The 7 Beginner Mistakes (and the Fixes)	19
08	Which Plan Should You Start On?	21
09	Staying Safe and Smart with Claude	23
10	Your 7-Day Claude Starter Plan	25

Intermediate		PAGE
11	Meet Claude Cowork	27
12	Cowork Workflows to Steal	29
13	Choosing Your Model: Opus, Sonnet, Haiku	31
14	Connecting Claude to Your Tools	33
15	Build Once, Reuse Forever: Your First Skill	35
16	Scheduling and Routines	37
17	Giving Claude a Memory: Context That Sticks	39
18	Build Your Own Assistant	41
19	Managing Usage, Cost, and Context	43
20	From Operator to Builder	45

Part 3 - Advanced

Claude Code, orchestration, and systems that run themselves.

Advanced		PAGE
21	Meet Claude Code: The Most Powerful Door	47
22	Claude Code Core Concepts	49
23	Creating Custom Workflows: Skills and Subagents	51
24	Orchestrating Agents at Scale	53
25	Extending Claude Code: Plugins, CLIs, and MCP	55
26	Integration Power-Ups: X + Y = Superpower	57
27	Building Systems That Run Themselves	59
28	The Business Operating System	61
29	Claude Code in the Real Business	63
30	Your Roadmap From Here	65

01

THE FOUNDATION

What Claude Actually Is (and Isn't)

The foundation every other chapter in this guide stands on.

BOTTOM LINE

Claude is a frontier AI assistant, not a chatbot and not a search engine. It reads, writes, reasons, codes, and can act on your behalf. Once you understand what it is genuinely good at and where it still needs a human, everything else gets easier.

01 Start here: it is not a chatbot

Most people meet Claude through a chat box, so they file it next to a search engine or a customer-service bot. That single framing is the biggest reason people underuse it. Claude is a general-purpose AI assistant built by Anthropic, an AI safety company. The same underlying intelligence can draft a proposal, analyze a messy spreadsheet, write and fix software, summarize a 200-page contract, or carry out a multi-step task on your computer while you do something else. The chat box is just the easiest door in.

02 What Claude is genuinely good at

- **Writing and editing.** Widely considered the strongest of the major models for natural, on-brand writing, long-form drafting, and tightening rough copy.
- **Reasoning and analysis.** It works through multi-step problems, reads long documents, and explains its thinking — holding up to a million tokens of context, roughly several thick books at once.
- **Coding.** Among the best in the world at writing and fixing software, scoring around 87 percent on a standard real-world engineering benchmark. You do not need to code to benefit.
- **Comparative accuracy.** In a 2026 long-form factuality test it made things up far less often than a leading competitor. Not flawless, but relatively careful.
- **Acting, not just answering.** Newer versions use tools, browse the web, work with your files, and complete tasks — not only return text.

03 One engine, many doors

Anthropic builds the intelligence — a family of models — then ships that same intelligence behind several doors, each shaped for a different person and a different level of independence. The chat website, a desktop app, a phone app, a coding tool, an autonomous desktop agent, and a developer interface all run the same brain. You choose the door by two things: who is doing the work, and how much you want Claude to act on its own.

04 What Claude is NOT

- **Not a search engine.** Trained on data up to a cutoff date, so it does not know today's news by default, though it can search the web when asked. Treat it as a reasoning engine, not a live index.
- **Not always right.** It can state wrong things with total confidence — hallucination. Verify anything that matters: names, numbers, quotes, sources.
- **Not deterministic.** Ask the same question twice and you may get two different answers. Normal for this kind of AI, and why high-stakes tasks eventually get structured.
- **Not a human, and not magic.** No memory of you between conversations unless you set it up, no instinct for your business unless you give it context, no judgment to spare you from checking its work.

WHY THIS MATTERS

The people who get the most out of Claude are not better at writing prompts. They understand what it is — a powerful, context-hungry reasoning engine — and what it is not, an oracle. They feed it context, they verify what matters, and they pick the right door for the job. That is the entire game.

■ Key takeaways

- 01 Claude is a frontier AI assistant from Anthropic — not a chatbot, not a search engine.
- 02 Strongest at writing, reasoning over long documents, and coding, and comparatively careful with facts.
- 03 One engine, many doors: the same intelligence reaches you through chat, desktop, mobile, a coding tool, an autonomous agent, and a developer interface.
- 04 Real limits — a knowledge cutoff, occasional confident mistakes, non-deterministic answers. Give it context and verify what matters.

02

THE MAP

The Claude Universe

Every way to use it

Six doors into the same intelligence, and how to know which is yours.

BOTTOM LINE

There is one Claude intelligence, reached through six different doors: the web chat, the Desktop app, mobile, Cowork, Claude Code, and the API. You do not need all of them. This chapter is the map, so the ones that follow make sense.

01 The six doors at a glance

THE DOOR	IN PLAIN WORDS	BEST FOR
Claude.ai (chat)	Claude in your web browser at claude.ai	Writing, analysis, brainstorming, quick tools
Desktop app	The same Claude as a Mac or Windows app	Daily use, plus one home for Cowork and Code
Mobile	The phone app, and a remote for your desktop	Quick questions and approving tasks on the go
Cowork	An agent that works on your desktop for you	Hands-off file, spreadsheet, and deck work, no code
Claude Code	Claude inside a terminal or code editor	Building and fixing software across a project
API	The developer plumbing behind Claude	Custom apps and automations that run unattended

02 The six doors, explained

01 **Claude.ai** — the front door

A website where you chat with Claude in your browser. Two features matter now. **Projects** are saved workspaces that remember your context and files, so you never start from zero.

Artifacts appear when Claude builds something interactive — a document, a table, a small app — in a side panel you can use and edit.

02 **Desktop app** — the daily home

The same Claude as a proper app on your Mac or Windows machine instead of a browser tab. Lighter than a browser, with a quick pop-up to reach Claude from anywhere, and it brings the more powerful modes (Cowork and Code) together in one place. If you use Claude every day, install this.

03 **Mobile** — the companion

The phone app. Good for quick questions on the go, and it doubles as a remote control: check on and approve longer tasks running on your computer, right from your phone. The companion, not the main workspace.

04 **Cowork** — the assistant that does the work

Claude as an autonomous assistant working on your actual desktop. Point it at a folder and give it a goal in plain English — “clean up these files,” “turn this into a slide deck” — and it does the work, asking permission as it goes. Built for non-technical people, no code or terminal. Covered fully in Intermediate.

05 **Claude Code** — the builder

Claude inside a developer’s environment, a terminal or code editor, that can build and fix software across many files on its own. It sounds technical, and it is the most powerful door, but you do not have to be a programmer to start. The heart of the Advanced level.

06 **API** — the plumbing

Not an app at all. It is what developers use to build Claude into their own products and to run automations that work unattended, with no person watching. Covered near the end of this guide.

03 Which door is for me?

Two questions guide the choice:

- **Who is doing the work?** If it is you — thinking and drafting with Claude helping — use the chat (web or Desktop). If you want Claude to go do something on your computer, use Cowork. If it is building software, that is Claude Code. If it is your own app or an automation that runs by itself, that is the API.
- **How much should it act on its own?** Low, where you steer every step, points to chat. High, where it runs and you review, points to Cowork, Code, or the API.

WHERE TO START

Most readers should live in Claude.ai chat and the Desktop app first; they cover the large majority of everyday needs. Add Cowork when you want work done for you, and Claude Code when you are ready to build.

■ Key takeaways

- 01 Same intelligence, six doors. Pick by who does the work and how much autonomy you want.
- 02 Start in Claude.ai chat and the Desktop app; they handle most beginner needs.
- 03 Cowork (Intermediate) does desktop work for you, Claude Code (Advanced) builds software, and the API powers unattended automations.
- 04 You can mix doors. A task often starts in chat and graduates to Cowork or Code as you scale it up.

03

THE MAIN DOOR

Claude.ai Deep Dive

Your first real workspace

Master the main door, the web chat, properly.

BOTTOM LINE

Claude.ai is where almost everyone starts, and where most everyday value lives. Learn five things — the interface, file upload, web search, Projects, and Artifacts — and you are already ahead of most users.

01 The interface, in 30 seconds

- **The conversation thread** sits in the center: you type at the bottom, Claude replies above. Start a new chat whenever the topic changes.
- **The model picker** near the top chooses which version of Claude answers. The default is fine for beginners.
- **The attach button** (a paperclip or plus) uploads files into the conversation.
- **Web search** can be toggled on so Claude looks up things newer than its training.
- **The side panel** opens on the right whenever Claude builds something interactive. That is an Artifact.

02 Uploading files

Drop in documents, spreadsheets, PDFs, images, and more, then ask Claude to work with them: summarize a long report, pull the key numbers from a spreadsheet, rewrite a contract clause in plain English, or describe what is in a screenshot. One of the fastest ways to get real value on day one. Instead of describing your document, just give it to Claude.

03 Web search: closing the knowledge-cutoff gap

Claude has a training cutoff and does not know today's news by default. Turning on web search lets it look up current information and cite where it came from. Use it for anything time-sensitive: recent events, current prices, a company's latest details. Leave it off when you just want Claude to reason over what you have already given it.

04 Projects: stop starting from zero

This is the single most valuable beginner feature, and most people never touch it. A **Project** is a saved workspace that remembers its context. You upload reference files once (up to around 20), set custom instructions once, and then every conversation you start inside it already knows your world. Think one Project per role, or per client.

A PLAIN CONVERSATION	A PROJECT
Forgets everything once the chat ends	Remembers context across every chat inside it
You re-upload your files each time	Files uploaded once, always available
You repeat your instructions every time	Instructions set once, applied automatically
Best for quick, one-off questions	Best for ongoing work for a role or client

Custom instructions are where you tell Claude who it is for this Project: your brand voice, your role, the format you want, the things to avoid. Set them well and you stop re-explaining yourself in every chat.

Projects are available on Claude's paid plans.

05 Artifacts: when Claude builds you a tool

When you ask Claude to make something more than text — a formatted document, a table, a chart, a simple calculator, even a small working web page — it opens an **Artifact** in the side panel. You can read it, use it, copy it, download it, and ask Claude to change it right there. It is the difference between Claude telling you how to build something and handing you the finished thing.

TRY

“Build me a one-page pricing sheet for three packages.”
“Make a simple ROI calculator I can fill in.”

DO THIS NOW**Build your first workspace**

- 01 Create a Project and name it for a real role or client.
- 02 Add custom instructions: who Claude is, your voice, what good output looks like.
- 03 Upload two or three reference files — a brand guide, past work, a key document.
- 04 Start a chat inside it and ask for something real. Notice you never had to explain your context. That is the upgrade from search bar to workspace.

■ Key takeaways

- 01 Claude.ai is the main door: a chat window that is really a workspace.
- 02 Upload files instead of describing them, and turn on web search for anything time-sensitive.
- 03 Projects remember your context, files, and instructions, so you stop starting from zero. Use one per role or client.
- 04 Artifacts hand you finished, editable things — documents, tables, charts, small apps — not just advice.

04

EVERY SCREEN

Desktop & Phone

Claude, everywhere you work

Get Claude off the browser tab and onto every screen you use.

BOTTOM LINE

The browser is fine for trying Claude. The Desktop app is where you should actually live — lighter, always a keystroke away, and the one place the more powerful modes appear later. The phone app keeps you connected and lets you approve work on the go.

01 The Desktop app: your daily home

The Desktop app is the same Claude, installed as a native application on your Mac or Windows machine instead of running in a browser tab. Three reasons to switch:

- **It is lighter and faster.** A browser tab can eat well over a gigabyte of memory. The app uses a small fraction of that, so it stays snappy even left open all day.
- **It is always a keystroke away.** A Quick Entry pop-up summons Claude over whatever you are working on — ask your question, dismiss it, no hunting for the right window.
- **It is the one home for everything.** The app brings chat, Cowork, and Claude Code together in one place, and can connect to apps and files on your computer.

Get it from claude.ai (look for the download link), install, and sign in with the same account. Your conversations and Projects follow you automatically.

02 The phone app: pocket assistant and remote control

The mobile app plays two distinct roles.

- **A pocket assistant.** Quick questions, dictating a draft while you walk, snapping a photo and asking about it, or capturing an idea before it disappears.
- **A remote control.** The part most people miss. If a longer task is running on your computer — a Cowork job or a Claude Code session — the phone lets you check on it, answer its questions, and approve its next step from anywhere. The feature for passing tasks between phone and desktop is called **Dispatch**.

One practical note: approval requests sent to your phone time out after about 30 minutes. It is built for staying in the loop while you step away briefly, not for leaving something hanging for hours.

03 When to use which

SURFACE

REACH FOR IT WHEN...

Web (browser)

Just trying Claude, on a shared or public computer, or you do not want to install anything.

Desktop app

Your daily driver — Claude a keystroke away, and you plan to use Cowork or Code.

Mobile app

On the go, capturing ideas, or checking and approving a task running on your computer.

DO THIS NOW

Install the Desktop app and sign in. Trigger the Quick Entry pop-up once so you know the shortcut. Then install the mobile app so you can capture ideas and approve tasks from anywhere. Now Claude is on every screen you use, all sharing the same account, conversations, and Projects.

■ Key takeaways

- 01 The Desktop app is the daily home: lighter than a browser, a keystroke away via Quick Entry, and the single place where chat, Cowork, and Code all live.
- 02 The phone app is both a pocket assistant and a remote control for tasks on your computer, through Dispatch. Approvals expire in about 30 minutes.
- 03 Your account syncs across all three surfaces — conversations and Projects included.
- 04 Rule of thumb: web to try it, Desktop to live in, mobile to stay connected.

05

THE CRAFT

Prompting Playbook

Six moves to better answers

Six moves that make every answer dramatically better.

BOTTOM LINE

Great results do not come from clever wording. They come from briefing Claude the way you would brief a sharp new hire: give it context, tell it exactly what you want, show it an example, and let it ask questions. Master these six moves and you will outperform most people using the same tool.

The biggest mistake beginners make is treating Claude like a vending machine: type a short request, expect a perfect answer. Treat it instead like a brilliant assistant on their first day — capable, but it knows nothing about you, your business, or what good looks like until you tell it.

01 Give it context — the biggest lever

Before the request, tell Claude what it needs to know: who you are, who the output is for, any relevant background. The same question with context attached produces a completely different answer. If the context lives in a document, upload it; if it is ongoing, put it in a Project so you never repeat yourself.

02 Ask it to ask you questions first

One line changes everything: **"Before you answer, ask me any questions you need to do this well."** Instead of guessing and handing you something generic, Claude surfaces the gaps in your request and fills them. This single habit prevents most bad first drafts.

03 Set the role and the format

Tell Claude who to be ("act as my marketing assistant," "you are a careful financial analyst") and exactly how you want the output — length, structure, tone. "A 120-word friendly email with one call to action" beats "write an email" every time. Vague in, vague out.

04 Show an example

If you have a sample of what good looks like — a past email you liked, a report in the right format — paste it in and say "match this style." Examples teach Claude your taste faster than any description, and stop output from sounding generic.

05 Iterate, do not restart

The first answer is a draft, not a verdict. Stay in the same conversation and steer: "shorter," "more direct," "drop the second paragraph," "warmer tone." Claude keeps the context and refines. Starting a brand-new chat throws away everything it just learned.

06 Make it check its work

For anything that matters, add "review your answer for mistakes before finalizing" or "double-check these numbers." Claude is comparatively careful, but it can still be confidently wrong. Asking it to verify catches a surprising number of errors — and you still confirm the important facts yourself.

07 Weak prompt vs strong prompt

WEAK

“Write a marketing email.”

STRONG

“You are my marketing assistant for a boutique accounting firm. Write a friendly cold email under 120 words to small-business owners offering a free bookkeeping review, with one clear call to action. Match the voice of this example: [paste]. Before you write, ask me anything you need.”

08 The reusable skeleton: R-C-T-F-E

Five parts, in order — then close with one request.

R	ROLE	who Claude should be
C	CONTEXT	the background and the files
T	TASK	exactly what you want done
F	FORMAT	length, structure, and tone
E	EXAMPLE	a sample of good output

Then close with: “Ask me questions before you start.”

■ Key takeaways

- 01 Brief Claude like a new hire, not a vending machine. Context is the single biggest lever.
- 02 Add “ask me questions first” to almost every real request; it kills generic drafts.
- 03 Set the role and format, and show an example to teach Claude your taste.
- 04 Iterate in the same conversation instead of restarting, and make it check its work on anything important.
- 05 Remember R-C-T-F-E: Role, Context, Task, Format, Example.

06

PUT IT TO WORK

10 Everyday Use Cases

Prompts you can use today

The fastest way to learn what Claude is for is to use it on something real.

BOTTOM LINE

Ten common use cases, each with a prompt you can paste in, adjust, and get value from today. Pick two or three from your own work and try them before the week is out.

01 Inbox triage

“I’ll paste a block of emails. For each one, tell me what action is needed, who it’s from, and the urgency (high, medium, low). Organize the output as a table.”

02 Summarize a long document

“Here is a document. Give me (1) a 3-sentence summary, (2) the 5 most important points as bullets, and (3) any action items or open questions. [paste]”

03 Plan a project

“I need to [describe the project]. Break it into phases with rough timelines. Ask me anything you need about scope, team, or deadline before you start.”

04 Prepare for a meeting

“I have a meeting with [who] about [topic] in [time]. Help me prepare: what to ask, what context to bring, and what outcome I should push for.”

05 Turn rough notes into a polished document

“Here are my rough notes from [context]: [paste]. Turn these into a clean, structured document in a professional tone. Keep the information, make it presentable.”

06 Research a topic quickly

“Give me a structured briefing on [topic]: what it is, why it matters, the key players or concepts, and the most important thing to know. Cite specifics so I know what to verify.”

07 Analyze a spreadsheet

“I’ll paste data from a spreadsheet. Identify the key trends, any anomalies or outliers, and what actions these numbers suggest. [paste]”

08 Draft and edit something

“Draft a [type] for [audience] about [topic]. Tone: [professional / friendly]. Length: under [number] words. Then: tighten it, cut the weakest sentence, and strengthen the opening line.”

09 Learn something fast

“Explain [topic] like I have a general business background but no technical knowledge here. Use a real-world analogy, then give me the 3 things I actually need to understand to use it.”

10 Make a decision

“I’m deciding between [A] and [B]. Context: [situation]. Give me a structured comparison: the top 3 pros and cons of each, which you’d recommend and why, and what would make me more certain.”

■ How to use this list

- **Do not try all ten at once.** Pick the two or three that match something you actually have to do this week.
- **Paste, then adjust.** Swap the parts in brackets, and add any relevant files or background above the prompt.
- **Iterate, do not restart.** After the first answer, steer it in the same conversation rather than starting fresh.

THIS WEEK

Complete at least two of these with real work from your own life. Notice how long it takes compared to doing the same task without Claude. That delta is your new baseline.

■ Key takeaways

- 01 The fastest way to learn Claude is to use it on something real, not to study prompting theory.
- 02 Adjust and iterate: the first output is a starting point, not a finished product.
- 03 Adding context — a document, your background, the intended audience — is the biggest upgrade to any of these prompts.
- 04 If something does not land, stay in the conversation and redirect instead of starting over.

07

AVOID THESE

7 Beginner Mistakes

And how to fix them

Most underuse of Claude traces back to the same few errors.

BOTTOM LINE

The difference between "Claude is okay" and "Claude changed how I work" almost always comes down to these seven habits. None of them require technical skill — just a shift in approach.

01 Blank-chat syndrome

Opening a new chat for every question, with no context, then wondering why the answers feel generic.

The fix. Build Projects (Section 3). Load your context once, set instructions once, and every chat inside that Project starts already knowing your world.

02 Vague asks

"Write me a proposal." "Summarize this." "Make it better."

The fix. Apply the R-C-T-F-E skeleton from Section 5 — Role, Context, Task, Format, Example. If you cannot brief a new hire in one paragraph, Claude cannot guess either.

03 Accepting the first answer

Getting a mediocre output, shrugging, moving on — or concluding Claude is not useful.

The fix. The first answer is a draft. Stay in the conversation and steer: "shorter," "more direct," "cut the second paragraph," "stronger opening." Iteration takes seconds.

04 No examples

Describing the tone in vague words like "professional but warm," and getting output that sounds nothing like you.

The fix. Paste an example — a past email, a piece you liked, a paragraph in the right voice. Showing beats telling every time.

05 The run-on prompt

One massive request that asks Claude to do seven things at once, with no structure.

The fix. Break complex tasks into steps in the same conversation, one thing at a time: "First an outline. Good, now section I. Good, now shorter." Sequential beats simultaneous.

06 Trusting blindly

Copying Claude's output straight into a proposal, email, or report without checking the facts or numbers.

The fix. Verify anything that matters — names, numbers, quotes, sources, legal claims. Add "check your work before finalizing" to high-stakes prompts. You are the editor.

07 Never using Projects

Re-uploading the same documents, re-explaining your brand voice, re-stating your role, in every single conversation.

The fix. One Project per role or client, set up once (Section 3). The single biggest quality-of-life upgrade for anyone using Claude more than a few times a week.

■ 30-second self-check

Before you send any prompt, ask yourself:

- ☐ Did I give it context, or am I starting blank?
- ☐ Is the task specific enough that a smart new hire would know what to do?
- ☐ Did I include an example of what good looks like?
- ☐ Is this going in a Project where the context already lives?

Four yeses and the output will almost always be usable. Any no — take 30 seconds to fix it before you hit send.

■ Key takeaways

- 01 The biggest mistakes are not technical — they are about context, specificity, and iteration.
- 02 Vague prompts get vague answers. The R-C-T-F-E skeleton fixes that.
- 03 Iterate inside the same conversation rather than starting over with a new chat.
- 04 Verify anything important. Claude is capable, but not infallible.
- 05 Projects eliminate most of the “I keep re-explaining myself” frustration.



CHOOSE WELL

Which Plan Do You Need?

Free · Pro · Max · Team

A clear framework so you stop over-paying or running out mid-month.

BOTTOM LINE

Four tiers: Free, Pro, Max, and Team / Enterprise. Most individuals using Claude seriously for work land on Pro. Max is for heavy daily users who hit Pro's limits. Team / Enterprise is for groups. Start lower and upgrade when you actually feel the friction.

01 Plans at a glance

PLAN	PRICE	WHO IT IS FOR
Free	\$0 / mo	Trying Claude, light occasional use
Pro	~\$20 / mo	Individuals using Claude daily for work
Max	~\$100–200 / mo	Power users who exhaust Pro's limits
Team	~\$25 / seat / mo	Shared access across a small team
Enterprise	Custom	Large organizations with compliance needs

Anthropic adjusts pricing and features. Check claude.ai for current figures before deciding.

02 What the plans actually differ on

- **Usage allowance.** How much you can do per rolling window (typically about five hours). Free is smallest; Pro is generous for most; Max is the highest.
- **Feature access.** Projects and some advanced features require a paid plan.
- **Model access.** Paid plans get priority access to the top models.
- **Team features.** Shared billing, admin controls, and user management at the Team and Enterprise tiers.

03 How usage windows work

Claude does not run on a monthly counter. It runs on a rolling window, typically around five hours. When you hit the limit, you wait for it to reset. Which means:

- **Bursts drain faster.** Heavy use in a short burst exhausts your allowance faster than the same usage spread across a day.
- **Even Max has ceilings.** Power users on the most intensive models still hit limits when they run very large workloads in concentrated bursts.
- **Habits help.** Usage-smart habits (Section 19) reduce how often you hit those walls.

04 Which plan to pick

Free — try it

Fine for occasional use, one-off questions, and trying Claude for the first time. You will hit the cap quickly if you use Claude seriously for work.

Pro — ~\$20/mo · daily work

The right starting point for most readers. Covers daily writing, analysis, Projects, and coding help. Most people with a genuine work use case should start here.

Max — ~\$100–200/mo · power user

Upgrade here when you are consistently hitting Pro's limits mid-project, not before. If you build with Claude Code for hours each day, this is where you end up.

Team — two or more people

Worth it as soon as two or more people use Claude regularly. Shared billing and centralized admin justify the slightly higher per-seat cost.

Enterprise — large organizations

When your organization needs SSO, data governance, compliance controls, and SLAs. Not a typical starting point.

THE RULE

Start on Pro. Use it intensively for 30 days. If you hit the usage ceiling more than once or twice a week, move to Max. If you never hit it, Pro is enough.

■ Key takeaways

- 01 Four practical tiers: Free (try it), Pro (daily work), Max (power user), Team / Enterprise (groups).
- 02 Claude usage runs on rolling windows, not monthly counters. Heavy bursts drain allowances faster.
- 03 The right plan for most individual daily users is Pro. Upgrade to Max when you feel the limit, not before.
- 04 Prices change. Always check claude.ai for the current numbers before deciding.



GOOD JUDGMENT

Safe & Smart

What to share, what to verify

What to share, what to keep out, and when to verify.

BOTTOM LINE

Claude is safe enough for most business use as long as you apply one principle: treat it like a capable contractor, not an employee under NDA. Sensitive information takes judgment, not paranoia — and everything consequential needs a human check before it goes live.

01 Safe to share vs keep out

SAFE TO SHARE

KEEP OUT

Work documents and reports

Passwords, API keys, secrets

Marketing copy and briefs

Personal data for people who have not consented

Your own strategic thinking

Client data under a confidentiality agreement
(check terms)

Research questions

Proprietary data with high competitive sensitivity
(use judgment)

General financial summaries

Specific client financial records (verify your
obligations)

The practical rule: if it would cause real damage appearing publicly, or there is a contractual reason it cannot leave your organization, keep it out or anonymize it.

02 How Anthropic handles your data

- **Default training.** On Free and Pro, conversations may be used to improve Claude's models. Turn this off in account settings under Data Controls.
- **Team & Enterprise differ.** Those plans have different retention and training terms. Read the applicable policies if you handle client or sensitive data.
- **It is cloud, not local.** Data goes to Anthropic's servers. You are not local-only unless you run a model on your own infrastructure — an advanced API case.

03 Verify what matters

Claude is comparatively accurate but not infallible. Match verification effort to the stakes:

- **Low stakes.** Brainstorming, drafting, internal summaries — use the output and move on.
- **Medium stakes.** External or client-facing content — read carefully and fact-check specific claims before sending.
- **High stakes.** Legal, medical, financial, anything permanent — treat Claude as a smart first draft and get a qualified human to review before acting.

04 When NOT to use Claude

- **Real-time data needed.** Claude has a knowledge cutoff. Use web search for current info, but verify anything critical from a primary source.
- **Final legal / medical / financial calls.** These need qualified professionals. Claude can help you understand or draft; the decision and accountability stay with a human.
- **When you cannot verify.** In a domain where you cannot tell if Claude is right, add expert review. Do not skip it because the answer sounds confident.

THE MINDSET

Claude is a high-quality first draft of almost everything. It saves you the blank-page problem and accelerates the first 80 percent. The last 20 percent – the part that goes live, signs a contract, or gives medical advice – stays with you.

■ Key takeaways

- 01 Treat Claude like a capable contractor: give it real work, but apply judgment about what to share.
- 02 Turn off training-data use in account settings if you handle sensitive material.
- 03 Verify proportionally: light for internal drafts, thorough for external or high-stakes output.
- 04 Some calls stay human: legal, medical, financial, and anything that cannot be undone.

10

ONE WEEK

Your 7-Day Starter Plan

Build the habit, then decide

Build the habit in one week. Then decide how deep to go.

BOTTOM LINE

The fastest way to embed Claude into your work is to commit one week to using it intentionally on real tasks. This is that plan. By the end you will know exactly how valuable it is for your work — and whether you want to go deeper.

01 The 7-day plan

DAY	FOCUS	WHAT TO DO
1	Set up	Install the Desktop app. Create your first Project. Set custom instructions.
2	Brief well	Take a real task you have today and prompt it with R-C-T-F-E. Compare the output to what you would have done without Claude.
3	Get wins	Use the prompts from Section 6. Pick the two or three that match your real work. Notice the time saved.
4	Files & search	Upload a document or spreadsheet you have meant to work through. Turn on web search and ask something that needs current info.
5	Build a tool	Ask Claude for an Artifact: a template, checklist, calculator, or one-page reference. Keep it.
6	Make it yours	Add a second Project for a different role or client. Refine the custom instructions based on what you noticed.
7	Review	Look back at the week. What saved the most time? What did not land? What would you automate next?

02 Your toolkit in one box

By the end of the week you will have:

- **Desktop app installed** with a keyboard shortcut to Quick Entry.
- **At least one Project** with real context and custom instructions.
- **A prompting habit** — R-C-T-F-E, ask questions first, then iterate.
- **A few reusable Artifacts** built from your actual work.
- **A clear sense** of where Claude saves you the most time.

03 Readiness check

You are ready for the Intermediate level when you can honestly say yes to all four:

- ☐ I use Claude at least three or four times a week on real work tasks.
- ☐ I have at least one Project set up and I know why it beats a plain conversation.
- ☐ I iterate inside conversations rather than restarting every time.
- ☐ I know what to verify, and I do it before high-stakes output goes anywhere.

Four yeses — move to the Intermediate level. It opens up Cowork, the right model for each job, connecting Claude to your other tools, and the Skills system that makes Claude truly yours.

ONE LAST BEGINNER THOUGHT

Most people stop here. They use Claude for writing help and feel a mild improvement. The people who get an order-of-magnitude upgrade are the ones who go further. The next two levels are where the real leverage is.

■ Key takeaways

- 01 One focused week builds a habit faster than months of occasional dipping.
- 02 The minimum viable setup: Desktop app, one Project, the R-C-T-F-E habit.
- 03 The readiness check tells you honestly whether to move on or consolidate first.
- 04 The Intermediate level is where Claude stops assisting you and starts working for you.

11

INTERMEDIATE

Meet Cowork

Claude stops answering and starts acting

Claude running as an autonomous assistant on your desktop.

BOTTOM LINE

You give Cowork a goal in plain English, it does the work, and it asks permission at key moments. No coding required. This is the jump from 'Claude helps me think' to 'Claude does the work while I think about something else.'

01 What Cowork actually is

In a normal chat you are in a back-and-forth conversation. Cowork changes that. You give Claude a goal, point it at your computer — a folder, a file, an app — and it executes step by step using tools to open files, read content, write outputs, and organize results. You see what it is doing and approve or redirect the decisions that matter.

02 The plan-to-action loop

1. Give Cowork a goal: 'Take all the feedback PDFs in this folder, extract the key themes, and build a summary spreadsheet — one row per document.'
2. Cowork builds a plan and shows it to you before doing anything.
3. You confirm the plan, or adjust it.
4. It executes step by step, flagging decisions that need your input.
5. You review the output at the end.

This loop keeps you in control without requiring you to do the work.

03 The sandboxed VM

Cowork operates inside a sandboxed virtual machine — a contained computer environment:

- **Isolated.** What it does in the sandbox does not affect your real computer unless you explicitly move something out.
- **Safe to experiment.** If something goes wrong, nothing is permanently broken. Reset and start fresh.

04 Ask Before Acting vs Act Without Asking

- **Ask Before Acting.** Cowork pauses before significant steps and waits for your approval. Best when you are learning, the task is new, or the stakes are high.
- **Act Without Asking.** Cowork completes the full task and only surfaces something if it genuinely cannot proceed. Best for routine, well-understood tasks you have run before.

Start with Ask Before Acting until you trust how Cowork handles your type of task.

05 Setting up Cowork

1. Open the Claude Desktop app.
2. Select Cowork from the mode options.
3. Point it at a folder you want to give it access to, or leave it at the default sandbox.
4. Describe your goal in plain English and confirm the plan.

GOOD FIRST TASK

'Take this folder of email threads, summarize the key decisions from each one, and put them in a table with columns for: thread subject, decision made, and next action.' Paste the folder path, run it in Ask Before Acting mode, and watch what it does. That 10-minute demo teaches more about Cowork than reading about it.

06 **What Cowork is good at vs its limits**

GOOD AT	LIMITS
File organization and renaming	Needs file access through paths you give it
Reading many documents and synthesizing them	Complex multi-app workflows work best on the API (Advanced section)
Building spreadsheets and reports from source material	Tasks requiring your real credentials or logged-in sessions need care
Batch operations on many files at once	

KEY MINDSET

Cowork is not magic. It is a precise, literal assistant. The better your goal is defined, the better the output. If you would not know how to explain the task to a new hire in two paragraphs, break it down further before giving it to Cowork.

■ **Key takeaways**

- 01 Cowork runs Claude as an autonomous assistant. You define the goal, it executes, you review.
- 02 The plan-confirm-execute-review loop keeps you in control without doing the work.
- 03 Start with Ask Before Acting. Switch to Act Without Asking once you trust the task.
- 04 The sandboxed VM means nothing permanent happens to your machine without your say.
- 05 Best for file work, batch processing, document synthesis, and spreadsheet generation.

12

COWORK IN ACTION

6 Real Workflows

Concrete tasks you can run today

Cowork is most valuable when work is repetitive, document-heavy, or involves many files at once.

BOTTOM LINE

Six workflows showing the range: from competitive research to monthly reporting. Adapt the ones closest to your actual work and run them on your own files today.

01 Competitor content scan

GOAL Find what a competitor is publishing and surface strategic gaps.

HOW Give Cowork a list of competitor websites or RSS links. Ask it to pull recent content titles and summaries, categorize them by topic, and flag themes you are not covering.

OUTPUT **A spreadsheet with competitor content mapped by topic, frequency, and angle - with a gap analysis column for your strategy.**

This would take a human researcher two or three hours of browsing. Cowork works through a structured list in minutes.

02 One-asset content repurposer

GOAL Turn one piece of long-form content into multiple formats.

HOW Give Cowork the source document. Ask it to produce: a LinkedIn post, a Twitter/X thread, a summary email, and three pull-quote candidates - all in your voice.

OUTPUT **Four ready-to-use pieces from one source document. Keep what lands, discard the rest.**

Most content gets one use. This workflow turns a single source document into five ready formats in one session.

03 Batch design brief

GOAL Take a content calendar and produce a structured brief for a designer.

HOW Give Cowork your content calendar. Ask it to write a one-paragraph creative brief per post: subject, hook, visual direction, and call to action. Output as a table.

OUTPUT **A table your designer can work from directly - no more per-post briefing back-and-forth.**

Writing a brief per post in separate conversations adds a day to every content piece. Batch mode removes it.

04 Morning brief

GOAL A daily intelligence document ready when you start work.

HOW Point Cowork at a folder where you drop news articles, newsletters, or bookmarks. Ask it to summarize each source, identify the two or three ideas most relevant to your work, and format the output as a one-page brief.

OUTPUT **A one-page reading each morning that replaces an hour of tab-switching.**

Tab-switching across five sources costs 30 to 45 minutes each morning. This converts that into a one-page read before you start.

05 Monthly financial dashboard

GOAL Turn a folder of monthly reports or CSV exports into a clear executive summary.

HOW Export your financial data to a folder (QuickBooks, Stripe, or a bank export). Ask Cowork to compile key figures, spot trends, flag anomalies, and produce a one-page summary table.

OUTPUT **A clean monthly snapshot. Verify the numbers yourself - the compilation and trend-spotting are done.**

The patterns in monthly numbers are obvious once compiled. The compilation is the friction point - this workflow removes it.

06 Automated invoicing

GOAL Generate a batch of invoices from a spreadsheet of client work.

HOW Give Cowork a spreadsheet with client name, project description, hours or deliverables, and rate - plus a template invoice. Ask it to produce one completed invoice per row.

OUTPUT **A folder of completed invoice documents, ready for your review and sending.**

Invoicing each client one at a time is the tax on every completed project. This batch approach reduces it to a final review.

07 Running these well

- **Define the output format upfront.** 'A spreadsheet with columns for X, Y, Z' is better than 'a summary.'
- **Start with Ask Before Acting** for any workflow you have not run before.
- **One task type at a time.** Combining too many steps produces messier results than chaining two or three focused instructions.
- **Review before using.** Cowork is fast, but you are the quality check.

THE COMPOUND EFFECT

Once a workflow is defined and working, you can re-run it on new input in minutes. That is where the value compounds - not the first run, but the tenth.

■ Key takeaways

- 01 Cowork's sweet spot is repetitive, document-heavy, batch work that would take a human hours.
- 02 Define the output format as precisely as the input. Vague output spec, vague output.
- 03 Chain focused instructions rather than bundling every step into one huge ask.
- 04 The value compounds: once a workflow runs, re-running it on new input takes minutes.

13

MODEL SELECTION

The Right Model

Speed, cost, and quality on every task

Haiku, Sonnet, Opus, or Fable. One of these fits each task.

BOTTOM LINE

Claude comes in four models, each with a different speed-intelligence tradeoff. The default (Sonnet) handles about 80 percent of tasks well. The other three cover the extremes. Master the pattern once and you will make a better call in five seconds.

01 The model family

MODEL	ANALOGY	TIER	BEST FOR
Haiku	The fast intern	Fast / Low	Volume tasks, quick summaries, real-time responses. Speed over depth.
Sonnet	The reliable senior	Fast / Mid	Most everyday work. The reliable default for 80 percent of professional tasks.
Opus	The expert specialist	Slow / High	Complex reasoning, nuanced judgment, high-stakes writing. Quality over speed.
Fable	The marathoner	Var / High	Very long documents, extended reasoning, tasks needing enormous context windows.

02 The simple rule

Start on Sonnet. Upgrade to Opus when the task demands it. Downgrade to Haiku when you need speed or volume. Consider Fable when the document or task is unusually large.

Most professionals using Claude for work operate on Sonnet the vast majority of the time and rarely need to change.

THE DEFAULT

If you never think about model selection, Sonnet is the right model to stay on. It is fast enough for conversation, smart enough for analysis, and cheap enough that you will not exhaust your allowance on typical work tasks.

03 When to change models

- **Output is missing nuance or making subtle errors.** Move up to Opus. The task needs deeper reasoning than Sonnet is applying.
- **You are processing high volume or need fast turnaround.** Drop down to Haiku. Let it handle the routine layer.
- **Your context is enormous and getting truncated.** Try Fable. It is built for very long documents and multi-step reasoning at scale.

04 The 70/20/10 pro pattern

Once you are using Claude intensively and thinking about cost, this is the sustainable routing pattern:

- **70% on Haiku.** Fast, routine work. Processing lists, quick summaries, first-pass drafts, formatting tasks.
- **20% on Sonnet.** Standard professional work. Analysis, solid drafts, client-facing output.
- **10% on Opus.** The hardest decisions. Complex strategy, nuanced editing, anything where quality genuinely moves the needle.

This pattern keeps cost manageable while reserving the best model for the work that benefits from it.

05 Token costs at scale

If you are building automations or running Claude at high volume through the API, the cost difference between models is significant. Haiku costs a fraction of Opus per token.

THE COST CASE

For automated pipelines processing thousands of items, routing to Haiku for routine steps and only escalating to Sonnet or Opus for decisions can cut costs by 80 percent or more. The logic applies even at modest scale.

06 The mistake to avoid

Two opposing failure modes. Both are common:

- **Using Opus for everything.** Slowest and most expensive for tasks that Sonnet handles equally well. You are paying the premium without the payoff.
- **Using Haiku for everything.** Misses the judgment, nuance, and quality that Sonnet or Opus bring to work that actually matters.

THE PRINCIPLE

The right model is the one that matches the task. Not the most powerful model you have access to. Not the cheapest. Match the intelligence level to the stakes and complexity of the work. That judgment costs nothing and saves a significant amount in time and money.

QUICK RULE

Sonnet for the work day. Haiku when you need volume or speed. Opus when you cannot afford to get it wrong or when the reasoning is genuinely complex. Fable when the document is enormous.

07 Task-to-model cheat sheet

1. Email replies, routine formatting, quick summaries, high-volume list processing = Haiku
2. Analysis, strategy documents, client-facing copy, code review, professional writing = Sonnet
3. Legal or financial analysis, complex debugging, nuanced judgment calls, high-stakes editing = Opus
4. Very long manuscript analysis, multi-step research across many documents, massive context tasks = Fable

■ Key takeaways

- 01 Four models: Haiku (speed and volume), Sonnet (reliable default), Opus (deep specialist), Fable (long-context marathoner).
- 02 Start on Sonnet. It handles about 80 percent of professional use cases well and rarely needs to change.
- 03 The 70/20/10 pattern (Haiku/Sonnet/Opus) is the sustainable, cost-efficient mix for intensive users.
- 04 Match the model to the task: Opus on routine work wastes money; Haiku on judgment calls wastes quality.
- 05 At API scale, routing by task type cuts costs by 80 percent or more. Automate the Haiku steps; reserve Opus for the decisions.

MODEL ROUTING

Model selection is a skill that pays back every day. Spend ten minutes deliberately routing your next twenty tasks and you will develop an instinct that sticks. After that it takes five seconds.

14

TOOL CONNECTIONS

Connecting Your Tools

Claude plugged into your work stack

Gmail, Calendar, Drive, Slack, Notion and more.

BOTTOM LINE

Claude becomes dramatically more useful when it can read from and write to the tools you already work in. This section covers the three ways to connect: built-in integrations, Claude in Chrome, and the Model Context Protocol.

BUILT-IN CONNECTORS Gmail, Calendar, Drive, Slack, Notion	CLAUDE IN CHROME Any tab, any site, no copy-paste	MCP Connect virtually any tool to Claude
---	---	--

01 Built-in connectors

CONNECTOR	WHAT CLAUDE CAN DO
Gmail	Read, draft, and send emails. Triage your inbox. Draft replies in your voice.
Google Calendar	See your schedule. Draft meeting agendas. Find free time slots.
Google Drive / Docs	Read documents and sheets. Update Docs. Save outputs directly.
Slack	Read channel history. Draft messages. Post updates to threads.
Notion	Read databases and pages. Write summaries back into pages.

These connect through the Integrations panel in Claude Settings. Once connected, Claude can pull from these sources automatically when you ask for context -- no copy-paste, no switching tabs.

ONCE CONNECTED

Claude can check your Gmail, see what is on your calendar, read a Drive doc, or pull from a Slack thread -- all within a single conversation. You do not need to describe what is already in those tools. Claude reads the source directly.

02 Claude in Chrome

The Chrome extension brings Claude into any browser tab with the page content already loaded:

- **Summarize an article** you are reading without copying it out of the tab.
- **Draft a Gmail reply** without switching to a separate Claude window.
- **Analyze a report** in Google Sheets or Docs without leaving it.
- **Get quick answers** about any page's content while staying in the tab.

The Chrome extension works with any website, not just Google tools. If the page is open, Claude can see it and work with it.

03 MCP: the deeper connection

The Model Context Protocol (MCP) is the standard that lets Claude connect to virtually any tool -- including ones Anthropic did not build. Think of MCP like USB-C for AI tools: a common plug that any compatible tool can use.

MCP servers exist for dozens of professional tools. If the built-in connectors do not include something you use, there is likely an MCP server for it already. If your team has a developer, they can build a custom MCP server for any internal tool in a few hours.

- **GitHub, Jira, Asana** for project and engineering workflows.
- **Salesforce, HubSpot** for CRM data and pipeline context.
- **Custom internal APIs** and databases your team already runs.

USB-C FOR AI

MCP is the standard connector that turns Claude into a hub for your entire stack. Any tool with an MCP server can talk to Claude. The list grows every month and most of the common professional tools are already covered.

04 What a connected week looks like

Four examples of how connectors change a working day:

- | | |
|-----------------------|---|
| MORNING | Summarize the 10 most important emails from the last 24 hours and tell me what needs a response today. (Gmail connector) |
| BEFORE MEETING | Read my calendar and draft a 3-point agenda for my 2pm client check-in based on the notes in the shared Drive doc. (Calendar + Drive) |
| AFTERNOON | Take the feedback from this morning's Slack thread and turn it into a structured brief for the design team. (Slack) |
| END OF DAY | Save today's decisions and action items to the project Notion page. (Notion) |

START WITH ONE

Pick the tool you use most -- Gmail, Drive, or Slack -- and connect it first. One integration that saves 20 minutes a day is worth more than five integrations set up and never used. Get value from one before adding more.

05 Setting up your first connector

1. Open Claude Settings and go to the Integrations panel.
2. Click the tool you want to connect and follow the authorization prompt. It takes about two minutes.
3. Start a new conversation, mention the tool by name, and ask Claude to pull context from it. The connection is live immediately.

■ Key takeaways

- 01 Built-in connectors (Gmail, Calendar, Drive, Slack, Notion) let Claude read from and write to your actual tools -- no manual copying needed.
- 02 The Chrome extension brings Claude into any browser tab with the page already loaded. It works on any site, not just Google tools.
- 03 MCP extends Claude to virtually any tool through a common standard. If the built-in list does not cover a tool you use, look for an MCP server.
- 04 Start with one connector. One integration used consistently beats five set up and abandoned.
- 05 A connected Claude does not need you to re-describe what is already in your tools. It reads the source directly, every time.

CONNECTED WORKFLOW

The biggest shift is not the time saved per task. It is that Claude stops being a separate tool you switch to and becomes part of the workflow you are already in. That changes how often you reach for it.

15

SKILLS SYSTEM

Your First Skill

Making Claude truly yours

Stop re-explaining yourself. Build once, reuse forever.

BOTTOM LINE

A Skill is a saved set of instructions, context, and behavior that Claude loads for a specific task. Build your first one and the upgrade is immediate: no more re-explaining your preferences, your audience, or your constraints.

THREE-TIMES RULE Do it 3x, then build a Skill	ROLE + CONTEXT + FORMAT What every good Skill must contain	CHAIN SKILLS Build a modular library for your ops
--	---	--

01 What a Skill actually is

A Skill is structured context that tells Claude who to be, what to know, and how to behave for a specific, repeatable task. It is closer to a detailed onboarding document for a specialist assistant than to an app or a plugin.

When you invoke a Skill, Claude loads everything in that document before responding. You never have to re-explain your preferences, your audience, your format, or your constraints again.

02 The three-times rule

Build a Skill when you have written similar setup instructions for the same type of task three or more times. The re-typing is the signal -- not the task itself, but the friction of re-explaining it.

THE RULE

If you have asked Claude to write a client update in your voice, keep it under 150 words, and reference project status -- more than twice -- that instruction set is a Skill. Write it once. Never re-type it.

03 How to build one

- **Describe it.** Tell Claude: 'I want to build a Skill for [task]. It should always do [behavior], know about [context], and produce [format]. Help me write it.' Claude drafts the Skill for you.
- **Interview method.** Tell Claude: 'I need a repeatable workflow for [task]. Interview me to understand what always has to be true, what context matters, and what good output looks like.' More thorough -- better for complex tasks.

Either method produces a working Skill in under 20 minutes. Choose Describe It for simple tasks; use the Interview method for workflows with many variables.

WHERE SKILLS LIVE

Save Skills as documents inside a Claude Project. When you start a conversation in that Project, the Skill is already present -- invoke it by name and Claude loads it immediately.

04 What a good Skill contains

- **Role.** Who Claude is in this context and the specialist mindset it brings.
- **Standing context.** Background knowledge that never changes: your brand, your process, your clients.
- **The repeatable task.** Exactly what Claude does each time the Skill is invoked.
- **Output format.** Structure, length, tone, and what to avoid.
- **At least one example.** A real past output that landed well. The fastest way to lock in your taste.

05 Chaining Skills

Skills can reference other Skills, letting you build a library of modular, composable pieces. A single prompt can call both a Brand Voice skill and a Client Summary format skill at the same time.

MODULAR LIBRARY

Use the Brand Voice skill and the Client Summary format from the Client Update skill to draft this status report. This is how you turn Claude from a general assistant into something that genuinely knows your operation.

06 Worked example: Weekly Client Update

CONTEXT	You are an assistant for a digital agency. Keep updates under 150 words. Confident, friendly tone. No jargon. Cover what was done, what is next, and any blockers.
TASK	When given a project name and a set of raw notes, produce a client update ready to send.
FORMAT	Three short paragraphs. No subject line. No bullet points.

ONCE BUILT

Workflow becomes: 'Client update, Project X, notes: [paste raw notes].'
Output is ready in under a minute. The Skill holds every constraint so you never re-type them.

The same pattern applies to any recurring deliverable: pitch emails, design briefs, project scopes, status reports.

KEEP SKILLS ALIVE

A Skill that does not reflect your current process produces confidently wrong output. Treat Skills like living documents: update them when your format or examples change. A quarterly 20-minute review keeps your whole library sharp.

07 Quick Skill checklist

1. Open a Project and create a new document. Name it after the Skill.
2. Write who Claude is in this context in one sentence.
3. Add the standing context: everything Claude should always know.
4. Define the repeatable task and the output format explicitly.
5. Paste one real example of ideal output at the bottom.

■ Key takeaways

- 01 A Skill is structured context that Claude loads for a specific repeatable task. It is not code or a plugin.
- 02 Build one when you have done the same setup work three or more times.
- 03 Good Skills contain: role, standing context, the repeatable task, the output format, and at least one real example.
- 04 Chain Skills to build modular, composable workflows -- Brand Voice plus Client Format in a single prompt.
- 05 Keep Skills current: an outdated Skill is worse than no Skill. Review them quarterly.

YOUR SKILL LIBRARY

One Skill turns a repeated task into a one-line prompt. Five Skills turns Claude into a system that knows your operation. That is where the real leverage is.

16

SCHEDULING & ROUTINES

Claude on Autopilot

Put the work that repeats on a schedule

Local schedule or cloud routine -- it just happens.

BOTTOM LINE

The work that repeats every day or every week is a candidate for automation. Claude can run scheduled tasks locally on your computer and in the cloud. This section covers the two types, what to automate vs leave manual, and how to stay human-on-the-loop.

LOCAL SCHEDULE	CLOUD ROUTINES	HUMAN-ON-THE-LOOP
Runs on your machine via Cowork	Runs on Anthropic's servers, anytime	You review outputs, not do the work

01 Two types of scheduling

TYPE	HOW IT WORKS	BEST FOR
Local schedule (Cowork)	Claude Desktop runs the task on your computer at a set time, while your machine is on.	Tasks needing local files, your apps, or your desktop state.
Cloud Routines	Claude runs in the cloud on Anthropic's servers, on a schedule, without your computer being on.	Tasks that are purely digital and do not need your local machine.

02 Setting up a local schedule

1. Build and test the task once manually. Confirm it produces the output you want.
2. Open the schedule settings and set the recurrence: daily at 8am, every Monday, the first of the month.
3. Set the output destination: a folder, a file, a location it can always write to.
4. Enable Ask Before Acting or Act Without Asking depending on how much you trust the routine.

Best for: morning briefings from a local folder, weekly file processing, recurring reports from spreadsheets on your machine.

03 Setting up a Cloud Routine

- Use the **/schedule command** in Claude Code or the Routines panel in Claude Desktop.
- Define **what to do, when to run it**, and where the output should go: a cloud folder, an email draft, a Notion page.
- It runs **whether your computer is on or not**. No machine dependency, no manual trigger.

Best for: sending a weekly digest to your inbox, generating a Monday morning brief from web sources, any task that does not depend on your local files.

04 What to automate vs leave manual

AUTOMATE

- Recurring summaries and briefings: daily, weekly, monthly
- Batch file processing on the same folder
- Regular reports from the same data source
- Reminders and check-ins that follow a fixed schedule

LEAVE MANUAL

- Tasks where the input changes unpredictably: you need to judge what to give it
- Anything requiring real-time decisions you want to make yourself
- High-stakes outputs that go directly to clients without your review

05 Running routines well

- **Review outputs regularly.** Run a new routine daily for the first two weeks and check every output. Catch drift early.
- **Set a quality gate.** For anything that goes to a client or gets sent externally, keep a manual review step before it leaves your control.
- **Log failures.** Configure the routine to notify you if it fails or produces no output. A silent failure is worse than an obvious one.
- **Update the routine when your process changes.** Stale routines produce stale outputs. Treat them like Skills: living documents.

THE MINDSET

Automation without oversight is just scheduled mistakes. Stay on the loop, not in it. Human-on-the-loop means you review and approve -- not that you do the work. That is the right level of automation for most people.

YOUR FIRST ROUTINE

Start with the task you do manually every single week without fail. A Monday morning brief, a Friday status summary, a daily folder digest. Build it once manually in Cowork, confirm the output is right, then set it to run on schedule. That single routine saves 30 minutes a week and proves the model before you build anything more complex.

THE TWO-WEEK RULE

Any new routine should be reviewed every output for the first two weeks. This is when drift appears -- the task runs, but the output no longer matches what you actually want. Catching it early costs 10 minutes. Catching it late costs a client or a week of bad reports.

06 Signs a routine needs updating

1. The output format has drifted from what you originally defined.
2. Your underlying process has changed and the routine does not reflect it.
3. The data source it reads from has changed structure or location.
4. You are editing the output manually every time -- that is the signal to update the prompt.
5. The routine is failing silently: it runs but produces no useful output.

■ Key takeaways

- 01 Two scheduling types: local (Cowork -- needs your machine on) and cloud (Routines -- runs on Anthropic's servers, anytime).
- 02 Automate what repeats with a predictable input. Leave manual what requires your judgment each time.
- 03 Review automated outputs for the first two weeks of any new routine. Catch problems before they become habits.
- 04 Human-on-the-loop means you review and approve, not that you do the work. That is the right level of automation for most people.
- 05 Stale routines produce stale outputs. Update them when your process changes -- treat them like living Skills.

SCHEDULED LEVERAGE

The goal is not to automate everything -- it is to remove the friction from the work that is already repeatable. One well-built routine that runs reliably every Monday morning is worth more than five routines set up and abandoned because they produced unreliable output.

17

CONTEXT AND MEMORY

How Claude Remembers

Why it forgets -- and what to do about it

Every new conversation starts blank. Here is how you solve that.

BOTTOM LINE

Claude has no persistent memory by default. Every new conversation starts blank. But there are three layers you can build to solve this: working memory, persistent memory, and a knowledge base. Understanding all three separates people who fight Claude's amnesia from people who have solved it.

WORKING MEMORY	PERSISTENT MEMORY	KNOWLEDGE BASE
Current conversation, clears when it ends	Projects and memory feature, lasts across sessions	Upload once, reference in every conversation

01 The three layers

Claude does not have a single memory system. It operates through three distinct layers, each serving a different purpose. Most people never learn this -- they fight Claude's amnesia instead of solving it.

Layer 1 is working memory: the current conversation. Layer 2 is persistent memory: Projects and the memory feature that survive across sessions. Layer 3 is a knowledge base: reference documents you upload that Claude can access whenever they are relevant.

02 Layer 1: Working memory

Working memory is the context window -- everything Claude can see in the current conversation. This includes your messages, its replies, any files you uploaded, and any Skills or system context loaded at the start. In the largest models it holds up to one million tokens. But it is temporary: it ends when the conversation ends, and the next one starts from zero.

CONTEXT ROT

As conversations grow very long, early parts fall out or get compressed. You will notice it when Claude stops referencing something it knew earlier, or when answers start to drift. Signs: repetition, contradictions, generic answers where it was once specific. Fix: start a new conversation and re-load the relevant context at the top.

03 Layer 2: Persistent memory

Persistent memory is how you make Claude remember across conversations. Two mechanisms, different purposes:

- **Projects.** Upload reference files and set custom instructions once. Every conversation inside that Project starts with that context already loaded. One Project per role or per client. This is the primary tool for persistent context.
- **The memory feature.** When enabled in account settings, Claude stores and retrieves facts about you across conversations -- your preferences, your role, recurring details. Best for personal facts; Projects carry the operational load.

Both are useful. They are not substitutes for each other: Projects carry operational context, the memory feature carries the personal layer.

ONE PROJECT PER ROLE

One Project per role: main job, active clients, research. Each conversation inside that Project starts with the context already loaded. Never re-explain it.

04 Layer 3: Knowledge base

For large, stable bodies of knowledge -- a full client brief, a product spec, a research archive, your SOPs -- the answer is not to paste it into every conversation. Upload it into a Project and let Claude access it whenever it is relevant. That is your knowledge base.

Good candidates for knowledge base documents:

- 1. Your brand guide and tone-of-voice document -- stable, referenced constantly.
- 2. A client's business overview -- shared context for every project with that client.
- 3. A research synthesis you reference repeatedly -- saves re-reading the source.
- 4. Your standard operating procedures -- keeps Claude aligned with how you work.

05 Signs of context rot

When a conversation runs long, performance degrades in recognizable ways. Catch these signs early and start a fresh session.

SIGN	LIKELY CAUSE	FIX
Generic advice on a topic Claude was specific about earlier	Early context compressed or dropped	Start fresh, re-load key context at the top
Repeats something it already produced	Long conversation, context drift	Restart with a summary of where you are
Contradicts itself between messages	Very long session, window overloaded	Break the task into shorter sessions

06 Let Claude help build your context

The fastest way to build a useful Project is to let Claude structure it. Describe your work and say: 'Help me identify what should go in a Project for this role -- what context, what custom instructions, what documents to upload.' Claude will produce a ready-to-use setup plan in one conversation.

THE AMNESIA IS SOLVABLE

Claude's blank-slate default is a design choice for privacy, not a flaw. But once you understand the three layers it is entirely solvable. One afternoon setting up a Project properly changes the daily experience permanently.

07 Quick setup checklist

1. Enable the memory feature in Claude settings if it is off.
2. Create a Project for your main role. Name it after the hat you wear.
3. Write custom instructions for that Project: your role, preferences, and how you like Claude to respond.
4. Upload two or three stable reference documents -- a brand guide, an SOP, or a client overview.
5. Run the interview method inside that Project to identify what else belongs there.

■ Key takeaways

- 01 Three layers: working memory (the conversation), persistent memory (Projects + memory feature), knowledge base (uploaded reference docs).
- 02 Context rot happens in long conversations. Watch for generic answers, repetition, or contradictions -- then start fresh.
- 03 Projects are the primary tool for persistent context. One per role or client. Every conversation inside starts with the context already loaded.
- 04 The memory feature stores personal facts. Projects store operational context. Both are useful; neither replaces the other.
- 05 Let Claude help you structure your Project. Describe your work, ask for a setup plan. It is faster than figuring it out alone.

THREE LAYERS, ONE SYSTEM

Working memory for now. Projects for each role. A knowledge base for stable material. Build all three and Claude stops being a tool you have to re-brief.

18

BUILD YOUR ASSISTANT

Build Your AI Assistant

The Four C's framework

The moment everything in this guide becomes one system.

BOTTOM LINE

The pieces are all in place: memory and context, tool connections, Skills, and routines. This section is about putting them together. The Four C's -- Context, Connections, Capabilities, Cadence -- are your blueprint for an assistant that knows your operation and shows up without being asked.

CONTEXT	CONNECTIONS	CAPABILITIES	CADENCE
It knows who you are and how you work	It works inside your actual tools	It executes tasks, not just answers	It shows up on schedule, not just when asked

01 The Four C's

Everything covered in the Intermediate level maps to one of four dimensions. Get all four right and you have an assistant that genuinely knows your operation -- not just a chat window you keep re-explaining your business to.

THE C	WHAT IT MEANS	WHERE YOU BUILT IT
Context	Claude knows who you are, what you do, and how you work	Section 17: Projects, custom instructions, reference docs, the memory feature
Connections	Claude works inside your actual tools	Section 14: Gmail, Calendar, Drive, Slack, Notion, MCP
Capabilities	Claude can execute tasks, not just answer questions	Sections 11-12 (Cowork), Section 15 (Skills)
Cadence	Claude shows up on a schedule, not just when you ask	Section 16: local schedules and Cloud Routines

A typical conversation delivers Context at best. All four together deliver something different: an assistant that runs without being asked.

02 What a fully-built day looks like

7:30 AM	Cloud Routine runs your morning brief. You open it on your phone over coffee. (Cadence)
9:00 AM	You start a client Project session. Claude already knows the client, the brief, and your voice. (Context)
10:30 AM	Draft an update for this client using yesterday's notes. Claude pulls from the Drive doc automatically. (Connections + Capabilities via Skills)
2:00 PM	You brief Cowork to process this week's feedback forms and build the summary sheet. You work on something else. (Capabilities via Cowork)
END OF WEEK	The weekly review Routine runs overnight and a summary lands in your inbox Monday morning. (Cadence)

03 The 4-step build

Build the Four C's in order. Each layer reinforces the next, and skipping Context first makes everything else harder.

1. Context first. Set up your core Projects and custom instructions. Upload your two or three most important reference documents. This is the foundation everything else inherits.
2. Add one connection. Connect the tool you use most -- Gmail, Drive, or Slack. Get real value from it for a week before adding the next one.
3. Build your first Skill. Pick the task you repeat most often and encode it. The weekly client update or a first-draft document are the classic starting points.
4. Add one routine. Identify a task that repeats on a fixed schedule and automate it. The morning brief is the standard first routine for good reason.

CONTEXT IS THE FOUNDATION

Every layer you add after Context -- connections, Skills, routines -- inherits that foundation automatically. Build it first and the rest compounds on top of it.

04 Start with one

The temptation is to build everything at once. Resist it. A single well-built Project that you actually use every day is worth more than five Projects that feel incomplete. One Skill you run three times a week beats ten Skills that sit idle.

Build the Four C's incrementally, starting with Context, then adding each layer as you actually feel the gap. The assistant you end up with is one you built by using it, not by planning it.

THE MEASURE

You stop thinking 'I need to explain my business to Claude' and start thinking 'Claude already knows my business.' That shift is what the Four C's produces when it is working.

05 Quick build checklist

1. Create a Project for your main role. Write custom instructions: who you are, how you work, what good output looks like for you.
2. Upload two or three stable reference documents -- a brand guide, your SOPs, or a key client overview.
3. Connect the one tool you use most. Gmail, Drive, or Slack. Use it for a full week before adding another.
4. Build one Skill for your most repeated task. The Skill that saves you the most re-typing is the one to build first.
5. Set up one routine. The morning brief -- a scheduled summary of what matters today -- is the standard starting point.

■ Key takeaways

- 01 The Four C's are Context, Connections, Capabilities, and Cadence. All four together produce an assistant that knows your operation and runs without being re-briefed.
- 02 Build in order: Context first. Everything else inherits the foundation you set in Projects and custom instructions.
- 03 Add one layer at a time. One connection, one Skill, one routine -- each used consistently before adding the next.
- 04 The measure of success is not how much you configured. It is whether you stop explaining your business and start just using the assistant.
- 05 Build incrementally. The assistant you actually use every day -- even a half-built one -- beats the fully planned system you have not started yet.

FOUR C'S COMPLETE

Context, Connections, Capabilities, Cadence. Build all four and Claude stops being a tool you switch to. It becomes part of how the work gets done.

19

USAGE, COST AND CONTEXT

Usage, Cost & Context Windows Explained

What you are actually paying for and how to get more from it.

BOTTOM LINE

Claude charges for tokens, not time. Every message re-sends the full conversation, so long sessions cost more than short ones. Understanding this -- and building habits around it -- lets you get significantly more value from the same plan without upgrading.

TOKENS	ROLLING WINDOW	CONTEXT COST	EFFICIENT HABITS
Unit of usage: ~4 characters each	5-hour reset, not monthly counter	Every message sends the full history	Get more from the same plan

01 How usage actually works

Claude does not bill by the minute or per question. It charges for tokens -- pieces of text roughly four characters each. Every time you send a message, the full conversation gets tokenized and counts toward usage.

The key insight: every message carries the full conversation cost. A 50-message conversation where each message is long can cost more in a single session than 10 short sessions. Context rot appears as conversations extend because the system compresses earlier content -- it does not forget randomly.

Usage on paid plans runs on a rolling window -- typically five hours -- not a monthly counter. Hitting the ceiling means waiting for that window to reset, not until next month.

02 What burns allowance fastest

- **Very long conversations:** Every message re-sends the entire history. A 50-message session costs more than five short ones with the same total content.
- **Large file uploads:** A 200-page PDF counts against the context window on every subsequent message in that session, not just when uploaded.
- **Running Opus for routine tasks:** Opus costs the most per token. Tasks that Haiku handles well at a fraction of the cost should not go to Opus.
- **Cowork and Code sessions with many tool calls:** Each tool call and result adds to the context. Long agentic sessions accumulate cost quickly.

03 Token-smart habits

1. Start a new conversation when the topic changes significantly. Old context you no longer need is pure cost with no benefit.
2. Summarize and compact long sessions. Ask Claude for a 300-word decision summary, then open a fresh session with that summary as context.
3. Upload files selectively. Paste the three sections you need rather than uploading a 300-page document and paying for it on every follow-up.
4. Route quick tasks to Haiku. Questions, first-pass summaries, and formatting work do not need Sonnet -- Haiku is faster and cheaper for these.
5. Use Projects for standing context. Project documents load once and are shared efficiently across sessions, unlike in-conversation uploads.

04 Upgrade vs get efficient

Before upgrading, ask whether efficiency habits would solve the same problem. Most usage issues are behavioral, not plan-limited.

GET MORE EFFICIENT IF...	UPGRADE IF...
You often carry old context you no longer need	You consistently hit the ceiling mid-project on work that matters
You use Opus for tasks Haiku would handle well	You run multiple long sessions every single day
You upload entire documents when you only need sections	You are building automations that need sustained high throughput
You rarely think about what goes into the context	Efficiency habits have not solved the problem after genuinely trying

05 Technical extra: context window sizes

Different models have different maximum context windows. Smaller models like Haiku have smaller windows. Opus and the top Sonnet variants support up to one million tokens -- roughly several thick books of text -- in a single context.

For most everyday use you will not come close to these limits. They become relevant when processing very large documents, running long agent sessions, or building specialized systems that need sustained context across many steps.

BY THE NUMBERS

~4 chars per token, on average	1 million token context -- Opus + top Sonnet	5 hours rolling usage window on paid plans
--	--	--

OPTIMIZE BEFORE YOU UPGRADE

More plan is not a substitute for smarter usage. Run the habits in Section 03 for a week -- new sessions when topics shift, Haiku for quick tasks, selective uploads. Upgrade only after that.

06 Model context at a glance

Different models suit different tasks. Matching the model to the job is one of the most direct ways to cut cost without losing output quality -- and it requires no plan changes.

HAIKU

Smaller window

Routine tasks: quick questions, summaries, formatting, first drafts

SONNET

Large window

Most everyday work: writing, analysis, coding, complex replies

OPUS

Up to 1M tokens

Long documents, complex sessions, best reasoning available

■ Key takeaways

- 01 Claude charges for tokens. Every message re-sends the full conversation, so long sessions cost proportionally more than short ones covering the same ground.
- 02 Usage runs on rolling windows, not monthly counters. A heavy burst within five hours can drain your allowance faster than spread-out use over the same period.
- 03 The biggest efficiency gains come from starting new sessions when topics shift, routing quick work to Haiku, and uploading only the sections of documents you actually need.
- 04 Projects are the most efficient way to maintain standing context. Project documents load once and are shared across sessions -- unlike in-conversation uploads that count against the window every time.
- 05 Upgrade when efficiency habits cannot solve the problem. Optimize first, then upgrade.

SMART USAGE BEATS BIGGER PLAN

The difference between a good session and an expensive one is almost never the model or the plan. It is knowing what belongs in the context and what does not. Build the habits in this section and the savings take care of themselves.

20

FROM OPERATOR TO BUILDER

From Operator to Builder

What's Next

You have graduated from user to operator. Here is what comes next.

BOTTOM LINE

By the end of the Intermediate level you are not just using Claude -- you are operating it. You have context set up, tools connected, Skills built, and some tasks on autopilot. The Advanced level takes you to builder: creating systems that run independently, using Claude Code, and integrating Claude into your products and workflows.

YOU'RE AN OPERATOR	ADVANCED LEVEL	READINESS CHECK	WHAT UNLOCKS
Context, tools, Skills and tasks on autopilot	Systems, Code, agents and products next	5 criteria to know when you are ready	Claude Code, SDK, business OS patterns

01 What you can do now

If you have worked through the Intermediate level, you can:

1. Maintain Projects that give Claude persistent context about your work, your voice, and your clients.
2. Connect Claude to the tools you use every day -- Gmail, Drive, Slack, Notion, and more via MCP.
3. Encode repeatable workflows into Skills and run them in seconds instead of prompting from scratch.
4. Delegate document-heavy tasks to Cowork and let Claude work through them while you do other work.
5. Schedule recurring tasks as local routines or Cloud Routines so they happen without manual triggering.
6. Choose the right model for each job and manage your usage efficiently across sessions and windows.

WHERE YOU STAND

That set of capabilities puts you in the top tier of professional Claude users. Most people stop here and get substantial, sustained value. That is a legitimate and productive place to land.

02 Your Intermediate toolkit in one box

- **Projects and custom instructions:** Context for who you are, how you work, and what matters most
- **File uploads and web search:** Answers grounded in real documents and live information
- **MCP connectors:** Gmail, Drive, Slack, Notion, GitHub -- Claude in your actual tools
- **Skills:** Repeatable workflows triggered in a single phrase
- **Cowork:** Autonomous task execution while you focus on other work
- **Cloud Routines and local schedules:** Recurring automation -- no manual triggering required
- **Model routing:** Haiku, Sonnet, or Opus -- matched to the job at hand

03 Readiness check: are you ready for Advanced?

Move to the Advanced level when you can check all five boxes below:

- ☐ You use Claude daily for real work -- not just occasionally or experimentally.
- ☐ You have at least two Projects and two Skills that you actually use on a regular basis.
- ☐ You have tried Cowork on at least one real task and seen it complete without you supervising.
- ☐ You have a recurring routine running -- something Claude handles automatically on a schedule.
- ☐ You want Claude to build things and run things for others, not just do things for you.

THE SHIFT

The Intermediate level is about making Claude work better for you. The Advanced level is about making Claude work for your users, your clients, or your business -- independently and at scale. If that is what you want, keep going.

04 What the Advanced level unlocks

Advanced is where Claude becomes infrastructure, not just a tool you reach for:

- **Claude Code:** The most powerful surface. Build and fix software, automate complex multi-file projects, and create systems that run without you present.
- **Orchestrating agents:** Run multiple Claude sessions in parallel, each working on a different part of a problem at the same time.
- **Plugins, CLIs, and MCP at depth:** Extend Claude's capabilities with tools from Anthropic, the community, and your own custom builds wired in via MCP.
- **The Agent SDK:** Build Claude into your own products and services with custom tools, permissions, and orchestration logic you control.
- **Claude as a business OS:** Use the architecture patterns from this guide to run an entire business operation with Claude as the backbone.

INTERMEDIATE LEVEL COMPLETE

You are now an operator. You have what most professional users never build.

■ Key takeaways

- 01 The Intermediate level makes you an operator: Claude is set up, connected, skilled, and partially running on autopilot. That is a real achievement -- most users never get here.
- 02 Most professional users stop at Intermediate and get strong, sustained value. That is a legitimate place to land. Do not feel pressure to go further before you are ready.
- 03 The Advanced level is for people who want to build systems, not just use them. Claude Code, agent orchestration, and the API are the primary tools for that next step.
- 04 The readiness check is honest: do not rush to Advanced. Solid Intermediate habits consistently produce more value than rushed Advanced experiments on an unsteady foundation.
- 05 If you want Claude to work for your users, your clients, or your business -- independently and at scale -- the Advanced level is waiting. Everything you built here is the prerequisite.

THREE LEVELS AT A GLANCE

BEGINNER Learn to use Claude well Prompting, history, plans	INTERMEDIATE Operate Claude effectively Projects, Skills, Cowork, Routines	ADVANCED Build systems with Claude Code, agents, API, SDK
--	---	--

THE FOUNDATION IS BUILT

Everything in the Advanced level builds on what you have here. Projects become product context. Skills become product features. Routines become automated pipelines. The vocabulary does not change -- the scale does.

21

MEET CLAUDE CODE

Meet Claude Code: AI That Builds The Most Powerful Door

And why you don't need to be a coder to open it.

BOTTOM LINE

Claude Code is Claude in a terminal or code editor, with the ability to read, write, and edit files across an entire project. It builds software, fixes bugs, designs systems, and runs automations. You do not need to be a programmer to start -- but it rewards people willing to engage at a technical level.

WHAT IT IS	NO CODE NEEDED	7 CAPABILITIES	4-STEP SETUP
Claude in your editor with full file access	Describe in English, Claude builds it	Code, fix, refactor, test, automate	Install and run in about 10 minutes

01 What Claude Code actually is

Claude Code is a command-line tool and IDE integration that gives Claude access to your file system, your terminal, and your codebase. Instead of responding in a chat window, it works inside your development environment -- reading files, writing code, running commands, and iterating on results.

The key difference from the chat interface: Claude Code can see and edit many files at once, run your code to test it, and make changes across an entire project -- not just answer questions about it.

02 You do not need to be a coder

That is not marketing language. Non-technical people use Claude Code to:

1. Build working prototypes from a plain-English description -- no syntax knowledge required.
2. Automate repetitive file operations and data processing tasks.
3. Create scripts that run on a schedule without any manual work.
4. Build internal tools such as dashboards, reports, and automators without a development team.

The pattern: describe what you want in plain English, review what Claude builds, test it, iterate. You are the product manager; Claude is the developer.

03 What it can do

- **Write code from scratch:** Describe what you want; it writes the code and runs it for you.
- **Read an existing project:** Point it at any codebase -- ask questions, add features, fix things.
- **Fix bugs:** Describe the problem or paste the error; Claude diagnoses and patches it.
- **Refactor and clean up:** Improve code structure and clarity without changing what it does.
- **Write tests:** Generate automated tests for existing functionality with no manual effort.
- **Build automations:** Scripts, pipelines, and scheduled tasks that run independently.
- **Work across many files:** Holds the full project in view and makes consistent changes everywhere.

04 Four-step setup

1. Install Node.js (free, at nodejs.org) if you do not already have it. It is the runtime Claude Code runs on.
2. Open a terminal. On Mac: Terminal app. On Windows: Command Prompt or PowerShell.
3. Run: **`npm install -g @anthropic-ai/claude-code`**
4. Navigate to your project folder, then run: **`claude`**

That is the complete setup. Claude Code opens in the terminal connected to your folder.

05 Your first run

Once Claude Code is running, try this:

1. Tell it what you want to build. For example: "Read all the CSV files in this folder, combine them, and output a summary." Plain English is enough.
2. Watch what it does: it reads your folder, proposes a plan, writes the code, and runs it. You see every step.
3. Review the output, test it, and iterate. Ask follow-up questions or request changes in plain English.

06 Golden rules for non-coders

- **Work in a test folder first:** Do not point Claude Code at your main business files until you understand how it behaves.
- **Read what it proposes before saying yes:** The plan step is your quality gate. Claude shows its work -- use that.
- **Save working versions:** Use Git or copy a working version before asking Claude to make changes.
- **Ask Claude Code to explain what it did:** "Walk me through what that script does" is a legitimate and useful instruction.

WELCOME TO ADVANCED

Claude Code is the first tool here. It is also the most powerful one.

YOU ARE THE PRODUCT MANAGER

Describe what you want in plain English, review the plan Claude proposes, test the output, and iterate. That relationship requires no code knowledge. Claude is the developer. You hold the brief.

■ Key takeaways

- 01 Claude Code is Claude in a terminal or editor, working across your full project. It reads, writes, runs, and iterates -- not just answers questions.
- 02 Non-technical users build real, working tools with it. Describe what you want, review the plan before approving, test the output, and iterate from there.
- 03 Setup takes about 10 minutes: install Node.js, run the npm install command, navigate to your folder, and type claude. That is the full process.
- 04 Start in a test folder. Read every plan before saying yes. Save working versions. Ask Claude to explain what it built. These four habits prevent almost every beginner mistake.

TERMINAL QUICK-START

- 01 **npm install -g @anthropic-ai/claude-code**
 - 02 **cd your-project-folder**
 - 03 **claude**
 - Claude Code opens. Tell it what you want to build.
-

THE CEILING JUST MOVED

Most people never get this far. The ones who do find that Claude Code closes the gap between knowing what they want and having it built. Start in a test folder. Run the command. See what happens.

22

CLAUDE CODE CONCEPTS

Claude Code Concepts

The Mental Model

Understand this once, and the rest of Claude Code makes sense.

BOTTOM LINE

Claude Code has a specific architecture: Workflows define the task, the Agent executes it, and Tools do the actual work. Understanding this triangle -- plus CLAUDE.md, context management, and permissions -- will save you hours of trial and error.

THE TRIANGLE	CLAUDE.MD	CONTEXT ROT	PLAN MODE
Workflows, the Agent, and the Tools	Standing instructions, read on every start	Long sessions drift, start fresh	Shift+Tab before risky tasks

01 The core mental model: Workflows, Agent, Tools

Everything Claude Code does fits one triangle. Get this, and the tool stops feeling like magic and starts feeling like a system you control.

- **Workflows:** The high-level plans Claude Code makes before acting -- the "here is what I am going to do" it shows you before executing. In Plan Mode, you see and approve the workflow first.
- **The Agent:** Claude itself, running in the terminal: reading and writing files, calling tools, and iterating on the results.
- **Tools:** What the Agent uses to actually do things -- read a file, write a file, run a command, search the codebase, call an API. Claude Code has built-in tools and can use external ones via MCP.

02 CLAUDE.md: your standing instructions

CLAUDE.md is a file you put in your project folder. Every time Claude Code starts there, it reads this file first. Think of it as the custom instructions from Projects, but for Claude Code. Use it to tell Claude Code:

1. What the project is and what it does.
2. What stack, libraries, and conventions to use, and how you want code formatted and named.
3. What it must not do -- delete certain files, change certain settings.
4. Context about the team or business that helps it make better decisions.

A good CLAUDE.md means you never re-explain the project. A missing one means Claude Code starts from scratch every single time.

03 The context window and context rot

Just like in chat, Claude Code operates within a context window. In very long sessions -- large codebases, many file reads, extended iterating -- the early context gets compressed, and Claude Code can lose track of earlier decisions or file contents.

- **Signs:** Inconsistent changes, forgetting about a file it saw earlier, or repeating a question it already asked.
- **Fix:** Start a fresh session with a short summary of what is done. Use CLAUDE.md to anchor the standing context so it never has to re-derive it.

04 Permissions and autonomy

Claude Code asks for permission before anything destructive or irreversible -- deleting files, running commands that modify your system. You control how much autonomy it has.

- **Default (ask each time):** Claude Code stops before any significant action and waits for your approval. Best for new projects and sensitive codebases.
- **Elevated trust:** Let it proceed on certain categories of action without asking each time. Only for well-understood, reversible operations.

Never give Claude Code blanket permission to do anything without understanding what that means in your specific project.

05 Plan Mode: Shift+Tab

Press Shift+Tab to activate Plan Mode. Claude Code builds the full workflow before taking any action, shows you exactly what it plans to do, and waits for approval. Use it for:

1. Large tasks touching many files.
2. Anything involving irreversible changes.
3. Any time you want to understand the approach before execution.

Use Plan Mode by default until you are comfortable with how Claude Code behaves in your specific project.

THE WHOLE TOOL IN ONE LINE

Workflows are the plan, the Agent is Claude doing the work, and Tools are how the work gets done. CLAUDE.md feeds it context, permissions keep it safe, and Plan Mode lets you look before it leaps.

■ **Key takeaways**

- 01 The mental model is a triangle: Workflows (the plan) + Agent (Claude) + Tools (file ops, commands, APIs, MCP).
- 02 CLAUDE.md is your standing instructions file. Create it on every project with /init and keep it current.
- 03 Context rot happens in long sessions. When changes get inconsistent or it forgets things, start a fresh session.
- 04 Permissions are granular -- start with ask-each-time and loosen only as you build trust. Plan Mode (Shift+Tab) is your safety net for large or risky tasks.

ESSENTIAL COMMANDS

/init	Read the project and generate a CLAUDE.md automatically
Shift+Tab	Toggle Plan Mode -- plan before acting
/btw	Add a note to the session context mid-task
Status line	Show the current task and progress in the terminal
Screenshot loop	Claude takes screenshots of running apps and iterates

GET THIS, AND THE REST IS EASY

Every advanced feature -- skills, subagents, MCP servers -- sits on top of this same triangle. Learn it once here, and the rest of the field guide stops feeling like magic and starts feeling like leverage.

23

SKILLS AND SUBAGENTS

Skills & Subagents

Delegate the Work

Encode your workflows once, then let Claude run them.

BOTTOM LINE

In Claude Code, Skills are structured markdown files that define repeatable behaviors, and subagents are separate Claude Code instances spawned to handle parts of a larger task in parallel. Together they let you run complex, multi-part work without constant supervision.

SKILLS	SUBAGENTS	TWO SKILL TYPES	PROGRESSIVE LOAD
Markdown files that define behaviors	Separate instances run in parallel	Capability and preference	Loads only what is relevant right now

01 Skills in Claude Code

Skills work on the same principle as the Intermediate Skills from Section 15, but they live as markdown files inside Claude Code and are loaded by the Agent when invoked. Instead of copy-pasting a prompt each time, the Agent reads the Skill file and executes it. A good Skill contains:

- **Role and capability:** Who Claude Code is acting as -- a code reviewer, a data analyst, a documentation writer.
- **The task definition:** What it should do when invoked, specific enough that the output is consistent.
- **The output format:** What the result looks like -- a file, a formatted report, a changed codebase.
- **Constraints:** What it must not do, and what to ask before doing.
- **An example:** The single most useful thing you can add.

02 Two types of Skill

- **Capability skills:** Define what Claude Code can do -- "when asked to review code, always check these five things and output a structured report."
- **Preference skills:** Define how it should behave -- "always use TypeScript, never use any, prefer explicit error handling."

Progressive disclosure: Claude Code loads Skills only when relevant, not all at once, so the context window never fills with Skills you do not need. Make each one specific enough that Claude Code can tell when to use it.

03 Subagents: delegated workers

A subagent is a separate Claude Code instance that runs alongside the primary session, handling a delegated piece of work. The primary Agent spawns subagents for:

- **Parallel work:** Researching multiple things at the same time.
- **Isolated tasks:** Work that should not contaminate the main session context.
- **Specialization:** One subagent focused only on writing tests while the primary Agent implements.

Each subagent gets its own context window, its own tools, and its own instructions from the primary Agent. When it is done, it reports back.

04 Worked example: research-and-write pipeline

A primary Agent is tasked with producing a competitive analysis report.

1. It reads the Research Skill (how to gather information) and the Report Writing Skill (output format and depth).
2. It spawns three subagents -- one per competitor -- all researching in parallel.
3. Each subagent returns a structured summary to the primary Agent.
4. The primary Agent synthesizes the summaries and writes the final report using the Report Writing Skill.

What would take a person several hours, the primary Agent plus three subagents completes as one coordinated pipeline.

THE DISTINCTION IN ONE LINE

A Skill is a set of instructions for how to behave. A subagent is a separate instance that actually executes. Skills define the what and how; subagents do the work. They combine -- a subagent can carry Skills of its own.

SKILL	SUBAGENT
Instructions for how to behave	A separate Claude Code instance that executes
Loaded as context, no separate process	Runs in its own session, in parallel
Defines the what and the how	Does the actual work
Reusable across many sessions	Spawned for a task, then done

Key takeaways

- 01 Skills are markdown files that define repeatable behaviors -- capability (what to do) and preference (how to behave).
- 02 Progressive disclosure loads Skills only when needed, keeping the context window efficient.
- 03 Subagents are separate Claude Code instances delegated to run parallel or isolated work.
- 04 Skills plus subagents enable complex, multi-part workflows with minimal supervision.

24

ORCHESTRATING AGENTS

Orchestrating Many Agents

A Team, Not a Tool

When one Claude is not enough for the job.

BOTTOM LINE

Some tasks are too large, too parallel, or too complex for a single session. Orchestration coordinates multiple Claude Code agents, each working on part of the problem under a primary orchestrator. This is where Claude Code starts to feel like a team rather than an assistant.

ORCHESTRATOR	WORKTREES	/WORKFLOWS	COST X N
Holds the plan, coordinates workers	Isolated copies for true parallelism	Define multi-step pipelines	Each agent multiplies token usage

01 One agent to many

A single session has a context-window limit. Very large projects -- a full codebase refactor, a synthesis across dozens of sources, a multi-part content pipeline -- can exceed what one session can hold. Orchestration splits the work across agents, each holding a piece of the project in its context.

The primary Agent (the orchestrator) holds the overall plan and coordinates. The worker Agents -- subagents or parallel sessions -- execute specific pieces and report back.

02 Dynamic Workflows and Ultra Code

- **/workflows:** Define a multi-step pipeline Claude Code follows, each step potentially spawning subagents or handoff points. A flowchart Claude Code executes.
- **/ultracode:** A high-autonomy mode: hand it a complex task and it handles the breakdown, subagent spawning, and synthesis itself. For experienced users.

03 Worktrees for parallel sessions

A worktree is an isolated copy of your codebase Claude Code can work in without touching the main branch. Multiple sessions can each hold their own worktree, allowing genuine parallel development:

- **Session A:** Works on the authentication refactor in its worktree.
- **Session B:** Fixes the data import bug in its worktree.
- **Session C:** Writes tests for the existing feature in its worktree.

04 **Monitoring agents: the dashboard**

When running multiple agents, the agent dashboard (run `claude agents` in the terminal) shows the status of every running session: what each one is doing, whether any are waiting for input, and the output so far. All three from before run at the same time -- review and merge the ones that worked.

05 **Orchestrate, or keep it simple?**

USE ORCHESTRATION WHEN...	KEEP IT SIMPLE WHEN...
The task genuinely cannot fit in one context window	The task is straightforward and sequential
Different parts are truly independent	You are still learning how Claude Code works
You want parallel processing to save time	Coordination overhead would exceed time saved
You are building a system that needs to scale	It is a one-off that will not repeat

THE COST WARNING

Running multiple sessions multiplies your token usage -- each agent has its own context window. Five agents in parallel on a large codebase is five times a single session. Check your plan limits before running large orchestrated pipelines.

■ Key takeaways

- 01 Orchestration coordinates multiple agents on different parts of one large task.
- 02 The primary orchestrator holds the plan; worker subagents execute the pieces.
- 03 `/workflows` defines multi-step pipelines; `/ultracode` lets Claude Code handle the breakdown autonomously.
- 04 Worktrees enable genuine parallelism -- isolated copies of your codebase, one per session.
- 05 Monitor running agents via `claude agents`. Watch for usage multiplication at scale.

A TEAM YOU DIRECT

Orchestration is the moment Claude Code stops being one assistant and becomes a coordinated team. You hold the plan and the budget; the agents hold the pieces. Start small -- two parallel sessions -- before you scale to five.

25

PLUGINS, CLIS, MCP

Plugins, CLIs & MCP Servers

Extend the Defaults

The ecosystem that takes Claude far past its defaults.

BOTTOM LINE

Claude Code extends through three additions: Plugins (from the marketplace), CLIs (command-line tools it can call), and MCP Servers (connections to external systems). This is how it reaches GitHub, browses real sites, controls a browser, or connects to your internal tools.

PLUGINS	CLIS	MCP SERVERS	GOLDEN RULE
Packaged tools from the marketplace	Terminal tools Claude calls as commands	Protocol connectors to external systems	Discover widely, install sparingly

01 Three extension types

TYPE	WHAT IT IS	HOW IT WORKS
Plugin	A packaged extension from the marketplace	Install once, Claude uses it when relevant
CLI	A command-line tool Claude Code can call	Install the tool, Claude calls it as needed
MCP Server	A server exposing a tool or data source via MCP	Run the server, Claude connects and uses it

02 Plugins worth knowing

PLUGIN	WHAT IT ADDS
Superpowers	Expanded capabilities pack for common tasks
Code-Simplifier	Suggests ways to reduce code complexity
Security Guidance	Flags security issues as code is written
Graphify	Creates visual diagrams from code or data
Codex	A library of code patterns and templates

03 CLIs that pair well with Claude Code

CLI	WHAT IT ENABLES
GitHub CLI (gh)	Read and write repos, PRs, issues, and actions
Playwright CLI	Browser automation: navigate, fill forms, scrape, test
Firecrawl CLI	Advanced web scraping with structured output
Google Workspace (gws)	Read and write Gmail, Drive, Docs, Sheets, Calendar
NotebookLM CLI	Programmatic access to Google NotebookLM

04 MCP servers: the deep connectors

MCP servers expose tools and data through a standard interface Claude Code uses natively. Notable examples:

- **Context7:** Up-to-date library docs, so Claude Code does not rely on outdated training data.
- **Composio:** A hub connecting hundreds of SaaS apps (Salesforce, HubSpot, Jira, Asana) through one MCP server.

05 Where to find them

- **Official extensions:** Listed in the Claude Code docs and the Desktop app's extension settings.
- **MCP registry:** The Anthropic GitHub org and community repositories list available servers.
- **npm and pip:** Many CLIs are standard packages with Claude Code integration.

DISCOVER WIDELY, INSTALL SPARINGLY

Every extension adds to the context Claude Code loads at startup. Too many slow things down and can conflict. Know what exists, but install only what earns its place.

■ The practical approach

1. Know what is available in the ecosystem -- the survey on the previous pages covers the most useful.
2. Install only what you use regularly.
3. Add a new extension when you have a specific task it solves, not speculatively.

■ Key takeaways

- 01 Three extension types: Plugins (marketplace tools), CLIs (terminal tools Claude calls), MCP Servers (protocol connectors).
- 02 The most useful CLIs: GitHub, Playwright, Firecrawl, Google Workspace, NotebookLM.
- 03 MCP extends Claude Code to hundreds of SaaS tools via Context7, Composio, and custom servers.
- 04 Discover widely, install sparingly. Each extension adds startup overhead; keep only what you use.

26

INTEGRATION POWER-UPS

Integration Power-Ups

Claude + Your Stack

What happens when you combine Claude with the right tools.

BOTTOM LINE

Each integration is useful alone. The real leverage comes from combining them: Claude Code plus Firecrawl, plus NotebookLM, plus Playwright, plus Graphify and Obsidian. The pattern is always Claude as the intelligent layer on top of purpose-built tools.

FIRECRAWL Scraping pipelines at scale	NOTEBOOKLM Near-infinite cited memory	PLAYWRIGHT Real browser automation	GRAPHIFY + OBSIDIAN A living second brain
--	--	---	--

01 Claude Code + Firecrawl = scraping at scale

- **Unlocks:** Crawl a site, extract structured data, and process the results automatically -- no brittle scraper code.
- **How it works:** Claude Code calls the Firecrawl CLI, which crawls and returns clean output. Claude Code then filters, categorizes, and summarizes it.
- **Real use case:** Point it at 50 competitor sites; extract each services, pricing, and about page into one comparison table, no manual browsing.

02 Claude Code + NotebookLM = near-infinite memory

- **Unlocks:** The context window has limits; NotebookLM holds a much larger body of knowledge and answers with citations.
- **How it works:** Claude Code queries a notebook via the CLI, gets a cited answer back, and uses it in the current task without loading everything into context.
- **Real use case:** Build a research-backed report: query the notebook for findings, get cited answers, synthesize a document without the full archive in context.

03 Claude Code + Playwright = browser automation

- **Unlocks:** Control a real browser -- navigate, fill forms, click, extract content, screenshot -- then decide based on what it sees.
- **How it works:** Claude Code drives Playwright through the CLI. Playwright launches a browser, performs the actions, and returns the results.
- **Real use case:** An automated intel loop: every Monday, browse 10 competitor sites, capture pricing or feature changes, and email you a diff report.

04 Graphify + Obsidian = second brain

- **Unlocks:** Turn Claude Code's outputs -- diagrams, notes, research, decision logs -- into a permanent, visual, linked knowledge base.
- **How it works:** Graphify generates visual diagrams from code or ideas; those plus session notes save into an Obsidian vault that grows into a linked graph.
- **Real use case:** Every session that produces a real decision saves to the vault with architecture diagrams -- an always-current technical knowledge base.

THE PATTERN BEHIND THE COMBOS

Claude reasons and decides, external tools execute the specialized work, and Claude synthesizes the results. This scales to any tool with a CLI or an MCP server -- the only question is what specialized tool handles a step better than Claude alone, and can Claude call it.

■ Key takeaways

- 01 Four high-value combos: Firecrawl (scraping), NotebookLM (large cited memory), Playwright (browser automation), Graphify + Obsidian (second brain).
- 02 The pattern is always Claude as the intelligent layer calling specialized tools for the specialized work.
- 03 Any tool with a CLI or an MCP server can join this pattern.
- 04 The real leverage comes from combinations, not individual tools.

27

SYSTEMS THAT RUN

Systems That Run Themselves

From Tasks to Production

From one-off tasks to reliable, unattended automations.

BOTTOM LINE

The earlier sections let you build things. This one is about making them run -- on a schedule, reliably, without you watching. Three deployment paths, one golden rule, and the guardrails every always-on automation needs.

LOCAL CRON Runs on your own machine	CLOUD ROUTINES Run on Anthropic's servers	AGENT SDK Production-grade systems	GUARDRAILS Hooks, permissions, secrets, logging
---	---	--	---

01 Three ways to deploy

METHOD	WHERE IT RUNS	BEST FOR
Local loop / cron	Your computer, on a schedule	Tasks needing your local files, apps, or credentials
Cloud Routine	Anthropic's servers	Purely digital tasks that do not need your machine
Cloud + Agent SDK	Your own cloud or hosting	Production systems, custom tools, client products

02 The local loop (cron)

A script runs on a schedule via your task scheduler (cron, Task Scheduler) or the `/schedule` command, calling Claude Code to do the task and save the output. Needs your machine on and awake. Good for morning briefs, weekly reports, and overnight local processing.

03 Cloud Routines

Covered in Section 16. These run on Anthropic's infrastructure, scheduled via the Routines panel -- you log in and review the outputs. Good for purely digital work: web research, email drafts, and content pipelines that use external APIs.

04 The Agent SDK: production-grade systems

The Agent SDK is the path for systems that run at scale, serve users, or power client-facing products. It is an API-based interface to embed Claude Code's intelligence into your own applications, with:

- **Custom tools:** Tools the agent can call, plus custom permissions defining what it is allowed to do.
- **Full orchestration:** Spawning subagents, routing tasks, and handling errors.
- **Production reliability:** Retries, logging, and monitoring. Managed Agents host it for you if you do not want to run a server.

05 Guardrails: keeping automations safe

- **Hooks:** Pre- and post-action scripts that validate what Claude Code is about to do, and can reject anything out of bounds.
- **Permissions:** Define exactly what the automation may access, write, send, or change. Keep it as narrow as possible.
- **Secrets:** Never hard-code keys or passwords. Use environment variables or a secrets manager.
- **Logging:** Every production automation logs what it did, when, and the result -- a record you can audit.

THE GOLDEN RULE

Automation built on the API runs reliably and at scale. Automation built on a subscription surface is for personal use and has no production SLA. Client-facing or always-on? Use the API and the Agent SDK. Just your own workflow? Subscription plus routines is fine.

■ Key takeaways

- 01 Three deployment paths: local cron (your machine), Cloud Routines (Anthropic's servers), Agent SDK (your cloud or Managed Agents).
- 02 Local is simplest but needs your machine on. Cloud Routines are hands-off but digital-only. Agent SDK is the production path.
- 03 Every production automation needs guardrails: hooks, narrow permissions, proper secrets, and logging.
- 04 Client-facing or always-on systems belong on the API, not on a subscription surface.

28

CLAUDE AS BUSINESS OS

Claude as Your Business OS

The Operating Layer

When Claude is not a tool you use but the backbone you run on.

BOTTOM LINE

At full depth, Claude is the operating layer your business runs through. This section is the architecture: how to structure files, decisions, and context so Claude has what it needs to run your operation intelligently across every function.

BUSINESS AS FILES A folder structure Claude navigates	FOUR C'S Context, Connections, Capabilities, Cadence	THREE GAPS Memory, consistency, accessibility	IT COMPOUNDS Build it once, gains stack up
--	---	--	---

01 The mindset shift

Most people use Claude as a tool they pick up when they need it. The Business OS model treats Claude as the system everything runs through: decisions logged in a format it can read, context structured so it can access it anytime, work organized in files it can find and edit. Instead of you re-explaining the business every time, the system carries that knowledge and Claude reads it.

02 Business as files: the architecture

FOLDER	WHAT LIVES THERE
context/	Who you are, what the business does, the team, priorities, goals
decisions/	A log of every significant decision: what, why, and the context
archives/	Old strategies and workflows. Nothing deleted, everything moved here
projects/	Active workstreams, each with its own folder and README
references/	SOPs, style guides, example outputs, research files
templates/	Reusable formats: proposals, briefs, onboarding, reviews

03 The Four C's at full depth

THE C	IN THE BUSINESS OS
Context	The context/ folder. Claude reads it each session and knows your business without you re-explaining.
Connections	MCP servers and CLIs that let Claude read and write your actual tools -- CRM, email, PM, analytics.
Capabilities	Skills in .claude/skills/. Every repeatable workflow the business runs.
Cadence	Scheduled routines: weekly review, morning brief, monthly snapshot, quarterly goal review.

04 The three gaps to close

1. Memory gap: Claude does not remember across sessions unless context is structured. Close it with the context/ folder and the decisions log.
2. Consistency gap: output varies because instructions vary. Close it with Skills -- every repeatable workflow encoded and standardized.
3. Accessibility gap: knowledge lives in heads or scattered files Claude cannot reach. Close it with an organized references/ folder and connected tools.

05 Build incrementally

- **Week 1:** Create the context/ folder with four or five files: who you are, what you do, your team, your priorities. Start each session with "read the context folder."
- **Month 1:** Add the decisions log. Start adding Skills for the workflows that repeat.
- **Quarter 1:** Add scheduled routines, connect your main tools, and build references for your most common work.

The architecture compounds. Six months of decisions logged in a standard format is a strategic asset. A library of tested Skills is a repeatable execution system.

■ Key takeaways

- 01 The Business OS treats Claude as the backbone of the operation, not a tool you pick up occasionally.
- 02 The architecture is a folder structure: context, decisions, archives, projects, references, templates.
- 03 The Four C's -- Context, Connections, Capabilities, Cadence -- map directly to that structure and to Skills.
- 04 Build incrementally, starting with the context folder. Close the three gaps: memory, consistency, accessibility.

29

REAL BUSINESS APPLICATIONS

Real Business Applications

Workflows That Pay

Three money workflows that prove Claude Code pays for itself.

BOTTOM LINE

The best argument for Claude Code is not capability, it is economics. Three real, revenue-connected workflows: building and delivering web properties, ranking in local search at scale, and running a content engine. Each is a service you can sell or a system that cuts your cost of delivery.

STITCH-TO-SITE Design then build, higher margin	LOCAL SEO Programmatic pages at scale	CONTENT ENGINE Publishing cadence, low marginal cost	THE PATTERN Process + Claude + tools + review
---	---	--	---

01 Build and sell websites with Stitch-to-site

A two-stage workflow for delivering professional websites faster and at higher margin than traditional design-and-build.

- **Stage 1 -- Stitch (design):** Generate and iterate a design system and page layouts from a brief. Client-ready visuals without a full design cycle.
- **Stage 2 -- Claude Code (build):** Hand the approved design over: "Build this as a Framer/Next.js/Webflow site, match the designs exactly, add a CMS."
- **The economics:** A two-week design-and-build compresses to two or three days. Same deliverable, higher margin, faster cash.
- **Sell it as:** Not "AI-assisted." The pitch is the same outcome in less time. The AI is the backend efficiency.

02 Rank locally with the 50k-clicks playbook

A programmatic local SEO system: generate hundreds of location- and intent-specific landing pages from a template and a data set, targeting every service-plus-location search in a market.

1. Define the template -- a service page optimized for local search intent.
2. Build the data set: services, service areas, and variations.
3. Let Claude Code generate every page from template plus data, then deploy.

What takes a content team months, it generates overnight. Sell it as a done-for-you build -- charge for strategy and setup; the generation is your margin.

03 Run a content engine

Claude Code produces blog posts, LinkedIn content, and social assets on a schedule in your voice. Define the strategy, build the source pipeline, let it draft to a review folder, then approve and publish. One session makes a week of publishable drafts; review drops from hours to minutes. Run it for your own brand, or package it as a recurring content retainer.

04 The underlying pattern

- **Defined process:** A well-defined, repeatable process is the foundation of every workflow.
- **Claude Code execution:** Claude Code is the intelligent layer that runs the process.
- **Specialized tools:** Stitch, the CMS, and search data handle the specialized steps.
- **Human review:** Quality control before anything goes live -- the margin protector.

THE PITCH IS NEVER "AI-ASSISTED"

In each case Claude compresses the production cycle while keeping the quality clients pay for. The margin improvement is the business case -- the AI is just how you get there.

■ Key takeaways

- 01 Three proven revenue workflows: Stitch-to-site (web delivery), local SEO at scale, content engine (publishing cadence).
- 02 Each one either reduces your cost of delivery or becomes a service you can sell.
- 03 The pitch is never "AI-assisted." The pitch is the outcome: faster, cheaper, same quality.
- 04 All three follow the same pattern: defined process + Claude Code execution + specialized tools + human review.

30

YOUR 90-DAY ROADMAP

Your 90-Day Roadmap

The Whole Guide, One Page

What you know, what to do next, and the one principle.

BOTTOM LINE

You have covered 29 sections across three levels. This is the compression: what you now know, what to do in the next 90 days, and the single principle that ties everything together.

THE CLIMB Beginner, Intermediate, Advanced	10 PRINCIPLES The whole playbook in one box	90-DAY PLAN Use it, systemize, then build	ONE IDEA Context is king
--	---	---	--

01 The climb you just made

LEVEL	WHAT YOU LEARNED	WHAT CHANGED
Beginner (1-10)	Claude, the interface, prompting, safe use, habits	Stopped treating Claude as a search engine
Intermediate (11-20)	Cowork, models, tools, Skills, routines, the Four C's	Started operating Claude as a system
Advanced (21-29)	Claude Code, subagents, orchestration, the Business OS	Started building systems that run without you

02 The whole playbook in one box

- 01 **Context is everything.** A well-briefed Claude beats a cleverly prompted one.
- 02 **One engine, six doors.** Match the door to the task and the autonomy you need.
- 03 **Projects stop you starting from zero.** One per role or client.
- 04 **Skills encode your workflows.** Three times means build a Skill.
- 05 **Cowork does the work; you review.** Hand off document-heavy and batch tasks.
- 06 **Match the model to the task.** Sonnet default, Haiku volume, Opus hard calls.
- 07 **Connect Claude to your tools.** No tools means half the value.
- 08 **Schedule the repeating work.** If it happens weekly, it should happen automatically.
- 09 **Claude Code builds things.** Software, automations, and systems live here.
- 10 **The Business OS compounds.** Context + Skills + routines get smarter over time.

03 Your 90-day plan

PHASE	FOCUS	WHAT TO DO
Use it well (1-30)	Foundations	Install Desktop, build your first Project, practice R-C-T-F-E, finish the 7-day starter, use it daily on real work
Make it yours (31-60)	Systems	Connect your tools, build two or three Skills for repeated tasks, set up one routine, try Cowork on a batch task
Build systems (61-90)	Scale	Set up Claude Code on a real project, build the Business OS context folder, encode core Skills, run one automated pipeline

04 Staying current

- **Follow the source:** Anthropic's changelog and announcements at anthropic.com.
- **Check when it feels old:** When something here feels outdated, use Claude's web search to verify the current state.
- **Refresh quarterly:** Revisit your Projects and Skills. Outdated context produces outdated output.

Context is king.

The people who get the most out of Claude are not the ones who know the cleverest prompts. They are the ones who give Claude the most useful context -- about who they are, what they do, what good output looks like, and what they are trying to accomplish.

The chat box is just the door. Everything behind it -- Projects, Skills, connections, CLAUDE.md files, the decisions log, the references -- is context. Build it well, maintain it, and Claude gets more useful every single week. That is the whole game.

WHERE TO GO FROM HERE

This guide is a foundation; the real learning comes from building. Pick one workflow from Section 29, one combo from Section 26, or one system from Section 28, and build it this week. The gap between reading about Claude and building with it is the only gap worth closing.

■ Key takeaways

- 01 The 30-section arc: foundations, then operating, then building.
- 02 The 90-day plan: 30 days using it well, 30 days making it yours, 30 days building systems.
- 03 Stay current -- the field moves fast. Check changelogs, verify with web search, refresh quarterly.
- 04 The one principle behind all 30 sections: context is king. Build it well and Claude compounds.

NEXUSPOINT

Want Claude doing this in your business?

Let's build the system.

You have seen what Claude can do across thirty sections. The gap between reading this and running it in your operation is where most people stall. That gap is exactly what we close.

FREE SYSTEMS AUDIT

Get a free teardown of your operation

We map the three highest-leverage places Claude can save you time or cut your cost of delivery. No pitch, no commitment, just a clear plan you can act on.

FOLLOW FOR MORE

New playbooks every week

Workflows, build breakdowns, and the systems we ship for real clients. Follow along and put the next one to work the day it drops.

START HERE

nexus-point.co